LAPORAN TUGAS BESAR 1
IF3170 INTELIGENSI ARTIFISIAL



Disusun Oleh:

BeraniDanTampan

Niko Samuel Simajuntak	13524029
Irvin Tandiarrang Sumual	13524030
Ray Owen Martin	13524033

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

DAFTAR ISI **2**
DAFTAR KODE **5**
DAFTAR GAMBAR **6**
BAB I
Deskripsi Persoalan **1**
1.1 Deskripsi Persoalan 1
BAB II
Pembahasan **2**
2.1 Implementasi Fungsi Pendukung Local Search 2
2.1.1 Inisialisasi Permasalahan 2
Kode 2.1 Implementasi metode __init__() 2
2.1.2 Representasi Geometri Paket 2
Kode 2.2 Implementasi fungsi _get_package_box() 3
Kode 2.3 Implementasi fungsi _get_boxes_per_truck() 3
Kode 2.4 Implementasi fungsi _boxes_overlap() 4
Kode 2.5 Implementasi fungsi _supports() 4
2.1.3 Pemeriksaan Constraints 5
Kode 2.6 Implementasi fungsi _is_package_within_bounds() 5
Kode 2.7 Implementasi fungsi _is_packages_overlapping() 6
Kode 2.8 Implementasi fungsi _is_package_floating() 6
Kode 2.9 Implementasi fungsi _is_fragile_supporting_another() 7
Kode 2.10 Implementasi fungsi _is_truck_overloaded() 7
Kode 2.11 Implementasi fungsi is_state_valid() 8
2.1.4 Perhitungan Objective Function 8
Kode 2.12 Implementasi fungsi calculate_objective_value() 8
2.1.5 Pembangkitan Initial State 9
Kode 2.13 Implementasi fungsi _candidate_coordinates() 9
Kode 2.14 Implementasi fungsi _find_feasible_placement() 10
Kode 2.15 Implementasi fungsi generate_initial_state() 11
2.1.6 Pembangkitan Neighbor State 11
Kode 2.16 Implementasi fungsi _swap_packages() 12
Kode 2.17 Implementasi fungsi _move_package() 12
Kode 2.18 Implementasi fungsi _rotate_package() 13
Kode 2.19 Implementasi fungsi generate_next_random_state() 14
Kode 2.20 Implementasi fungsi generate_best_neighbor_state() 15
2.2 Implementasi Algoritma Hill Climbing 16
2.2.1 Steepest Ascent Hill Climbing 16
2.2.1.1 Inisialisasi Algoritma 16

Kode 2.21 Implementasi metode __init__() 16
2.2.1.2 Algoritma Utama 17
Kode 2.22 Implementasi metode execute() 17
2.2.2 Sideways Hill Climbing 18
2.1.2.1 Inisialisasi Algoritma 18
Kode 2.23 Implementasi metode __init__() 18
2.1.2.2 Algoritma Utama 18
Kode 2.24 Implementasi metode execute() 18
2.2.3 Stochastic Hill Climbing 20
2.1.2.3 Inisialisasi Algoritma 20
Kode 2.25 Implementasi metode __init__() 20
2.1.2.4 Algoritma Utama 20
Kode 2.26 Implementasi metode execute() 20
2.2.4 Random Restart Hill Climbing 21
2.2.4.1. Inisialisasi Algoritma 21
Kode 2.27 Implementasi metode __init__() 21
2.2.4.2. Algoritma Utama 21
Kode 2.28 Implementasi metode execute() 22
2.2.5 Hasil dan Analisis Eksperimen 23
2.2.5.1 Hasil Eksperimen 23
2.2.5.2 Analisis Hasil Eksperimen 24
2.3 Implementasi Algoritma Simulated Annealing 25
2.2.1 Class SimulatedAnnealing 26
Kode 2.29 Pemanggilan method execute LocalSearch yang mengakses execute dari kelas SimulatedAnnealing yang memanfaatkan konsep polimorfisme 26
2.2.2 Class SimulatedAnnealingResult 26
Kode 2.30 Implementasi kode LocalSearchResult 26
Kode 2.31 Pemanggilan method visualize dari kelas LocalSearchResult yang memanfaatkan data dari SimulatedAnnealing dengan memanfaatkan konsep polimorfisme 27
2.2.3 Method __init__ pada Simulated Annealing 27
Kode 2.32 Implementasi kode __init__ pada Simulated Annealing 28
2.2.4 Method execute pada SimulatedAnnealing 28
Kode 2.33 Inisialisasi Data dan State 28
Kode 2.34 Membangkitkan neighbor 29
Kode 2.35 Evaluasi Objective Function 29
Kode 2.36 Menentukan next state 29
Kode 2.37 Bagian cooling algorithm 30
Kode 2.38 Pencatatan terakhir 31
2.2.5 Sejumlah Helper 31

Kode 2.39 Implementasi helper 31
2.2.6 Hasil dan Analisis Eksperimen 32
2.4 Implementasi Algoritma Genetic Algorithm 37
2.4.1 Inisialisasi Algoritma dan Populasi 37
Kode 2.40 Implementasi metode __init__() 38
Kode 2.41 Implementasi metode _initialize_population() 38
2.4.2 Perhitungan Fitness 38
Kode 2.42 Implementasi metode _fitness() 39
2.4.3 Seleksi Induk 39
Kode 2.43 Implementasi metode _select_parent() 39
2.4.4 Persilangan dan Perbaikan Individu 39
Kode 2.44 Implementasi metode _crossover() 40
Kode 2.45 Implementasi metode _repair_placements() 40
2.4.5 Mutasi Individu 41
Kode 2. 46 Implementasi metode _mutate() 41
2.4.6 Eksekusi Algoritma Genetic Algorithm 42
Kode 2.47 Implementasi metode execute() 42
2.4.7 Hasil dan Analisis Eksperimen 43
2.2.5.1 Pengaruh Jumlah Iterasi 44
2.2.5.2 Pengaruh Jumlah Populasi 45
2.2.5.3 Analisis Hasil 46
BAB III
Kesimpulan dan Saran 47
3.1 Kesimpulan 47
3.2 Saran 47
Pembagian Tugas 48
Referensi 49
Lampiran Form Penggunaan Generative AI 50
LAMPIRAN LAINNYA 52

BAB I
Deskripsi Persoalan

1.1 Deskripsi Persoalan

Persoalan yang diangkat dalam tugas besar ini adalah optimasi penempatan paket ke dalam peti kemas pada sebuah truk. Setiap paket memiliki karakteristik yang berbeda, meliputi dimensi (panjang, lebar, dan tinggi), nilai barang (*value*), berat (*weight*), status barang pecah-belah (*isFragile*), serta estimasi waktu kedatangan (*Estimated Time of Arrival* atau ETA). Sementara itu, peti kemas memiliki dimensi ruang dan kapasitas angkut maksimum yang terbatas. Dengan keterbatasan tersebut, tidak semua paket harus dimasukkan ke dalam peti kemas.

Tujuan utama dari persoalan ini adalah menentukan konfigurasi penempatan paket yang dapat memaksimalkan total nilai barang yang diangkut oleh truk. Konfigurasi tersebut mencakup pemilihan paket yang akan dimuat, penentuan posisi setiap paket dalam ruang tiga dimensi, serta orientasi paket. Setiap paket berbentuk kubus atau balok dan dapat memiliki hingga enam kemungkinan orientasi berdasarkan rotasi terhadap sumbu koordinat. Seluruh posisi paket direpresentasikan menggunakan koordinat bilangan bulat.

Dalam menentukan konfigurasi penempatan, terdapat beberapa batasan (*constraints*) yang harus dipenuhi. Paket yang dimuat harus berada sepenuhnya di dalam peti kemas sehingga tidak menembus dinding. Selain itu, paket tidak diperbolehkan melayang, yaitu berada pada posisi tanpa satu pun titik alas yang memiliki tumpuan, baik dari dasar peti kemas maupun paket lain. Paket yang tergolong barang pecah-belah juga tidak diperbolehkan menjadi tumpuan bagi paket lain. Di samping batasan geometris tersebut, total berat seluruh paket yang dimuat tidak boleh melebihi kapasitas angkut maksimum truk.

Persoalan ini diselesaikan menggunakan pendekatan *local search* untuk mencari konfigurasi penempatan dengan nilai fungsi objektif sebesar mungkin. Proses pencarian dimulai dari konfigurasi awal yang dibangkitkan secara acak, kemudian dilakukan perubahan konfigurasi melalui operasi pertukaran paket, pemindahan posisi paket, atau rotasi paket. Algoritma yang digunakan meliputi *Hill-Climbing*, *Simulated Annealing*, dan *Genetic Algorithm*. Ketiga algoritma tersebut diterapkan dan dievaluasi untuk membandingkan kualitas solusi yang dihasilkan, durasi pencarian, serta konsistensi hasil dalam menyelesaikan persoalan optimasi penempatan paket.

BAB II
Pembahasan

2.1 Implementasi Fungsi Pendukung *Local Search*

2.1.1 Inisialisasi Permasalahan

Inisialisasi permasalahan dilakukan melalui metode `__init__()` pada kelas `Problem`. Metode ini menerima dua parameter, yaitu `trucks` berupa daftar objek `Truck` dan `packages` berupa daftar objek `Package`. Kedua parameter tersebut digunakan untuk menyimpan informasi truk dan paket yang akan digunakan dalam proses pencarian solusi.

Kode 2.1 Implementasi metode `__init__()`

```
def __init__(self, trucks: list[Truck], packages: list[Package]):
    self.trucks = trucks
    self.packages = packages
    self.truck_map = {truck.id: truck for truck in trucks}
    self.package_map = {package.id: package for package in
packages}
    self.package_colors = self._generate_package_colors()

    if len(self.truck_map) != len(trucks):
        raise ValueError("Duplicate truck ID")
    if len(self.package_map) != len(packages):
        raise ValueError("Duplicate package ID")
```

Metode ini menyimpan daftar truk dan paket masing-masing dalam atribut `self.trucks` dan `self.packages`. Selanjutnya, dibentuk dua struktur data berupa *dictionary*, yaitu `self.truck_map` dan `self.package_map`, yang memetakan ID ke objek truk atau paket terkait. Penggunaan struktur data ini memungkinkan pengaksesan objek secara langsung berdasarkan ID tanpa harus melakukan pencarian secara berurutan pada daftar.

2.1.2 Representasi Geometri Paket

Representasi geometri paket digunakan untuk menentukan posisi dan ruang yang ditempati oleh setiap paket dalam peti kemas. Pada implementasinya, geometri paket direpresentasikan menggunakan tipe data `Box`, yaitu *tuple* yang terdiri atas enam bilangan bulat dengan format `(x_min, y_min, z_min, x_max, y_max, z_max)`. Tiga nilai pertama menyatakan

koordinat minimum, sedangkan tiga nilai terakhir menyatakan koordinat maksimum paket pada masing-masing sumbu.

Terdapat empat fungsi yang digunakan untuk mengelola representasi geometri paket, yaitu `_get_package_box()`, `_get_boxes_per_truck()`, `_boxes_overlap()`, dan `_supports()`.

- **Fungsi `_get_package_box()`**

Fungsi `_get_package_box()` digunakan untuk memperoleh batas koordinat tiga dimensi suatu paket berdasarkan posisi dan orientasinya.

Kode 2.2 Implementasi fungsi `_get_package_box()`

```
def _get_package_box(self, package_id: str, placement:
PackagePlacement) -> Box:
    coordinate = placement.coordinate
    if coordinate is None:
        raise ValueError("An outside package does not have a
box")
    dimensions =
placement.orientation.get_oriented_dimensions(self.package_map[pa
ckage_id].dimension)
    return (coordinate.x, coordinate.y, coordinate.z,
coordinate.x + dimensions.w, coordinate.y + dimensions.l,
coordinate.z + dimensions.h)
```

Fungsi ini menerima ID paket dan objek `PackagePlacement` yang memuat informasi posisi serta orientasi paket. Apabila paket berada di luar truk sehingga tidak memiliki koordinat, fungsi akan menghasilkan pengecualian `ValueError`. Selanjutnya, dimensi paket disesuaikan dengan orientasinya melalui metode `get_oriented_dimensions()`. Hasil akhirnya berupa `Box` yang menyatakan batas minimum dan maksimum paket pada sumbu x, y, dan z.

- **Fungsi `_get_boxes_per_truck()`**

Fungsi `_get_boxes_per_truck()` digunakan untuk mengelompokkan representasi geometri seluruh paket yang berada di dalam truk berdasarkan ID truk masing-masing.

Kode 2.3 Implementasi fungsi `_get_boxes_per_truck()`

```
def _get_boxes_per_truck(self, state: State) -> dict[str,
dict[str, Box]]:
    boxes = {truck_id: {} for truck_id in self.truck_map}
    for package_id, placement in state.placements.items():
        if placement.is_inside:
            boxes[placement.truck_id][package_id] =
```

```
self._get_package_box(package_id, placement)
    return boxes
```

Fungsi ini menerima sebuah `State` yang merepresentasikan konfigurasi penempatan seluruh paket. Pada awalnya, dibentuk *dictionary* kosong untuk setiap truk. Selanjutnya, fungsi melakukan iterasi terhadap seluruh paket dan memeriksa atribut `is_inside` untuk menentukan apakah paket berada di dalam truk. Paket yang berada di dalam truk akan dikonversi menjadi `Box` menggunakan fungsi `_get_package_box()` dan dikelompokkan berdasarkan ID truknya. Hasil pengelompokan ini digunakan untuk mempermudah pemeriksaan hubungan geometris antar paket dalam truk yang sama.

- **Fungsi `_boxes_overlap()`**

Fungsi `_boxes_overlap()` digunakan untuk memeriksa apakah dua paket menempati ruang yang saling bertumpang tindih dalam ruang tiga dimensi.

Kode 2.4 Implementasi fungsi `_boxes_overlap()`

```
@staticmethod
def _boxes_overlap(first: Box, second: Box) -> bool:
    return (
        first[0] < second[3] and second[0] < first[3]
        and first[1] < second[4] and second[1] < first[4]
        and first[2] < second[5] and second[2] < first[5]
    )
```

Fungsi ini menerima dua objek `Box` dan mengembalikan nilai `Boolean` yang menunjukkan apakah keduanya saling bertumpang tindih. Pemeriksaan dilakukan dengan membandingkan batas koordinat kedua paket pada masing-masing sumbu x, y, dan z. Dua paket dianggap bertumpang tindih apabila terdapat irisan dengan panjang positif pada ketiga sumbu secara bersamaan.

- **Fungsi `_supports()`**

Fungsi `_supports()` digunakan untuk memeriksa apakah suatu paket dapat menjadi penopang bagi paket lain yang berada di atasnya.

Kode 2.5 Implementasi fungsi `_supports()`

```
@staticmethod
def _supports(bottom: Box, top: Box) -> bool:
    return (
```

```
        bottom[5] == top[2]
        and bottom[0] < top[3] and top[0] < bottom[3]
        and bottom[1] < top[4] and top[1] < bottom[4]
    )
```

Fungsi ini menerima dua objek `Box`, yaitu `bottom` sebagai kandidat paket penopang dan `top` sebagai paket yang berada di atasnya. Pemeriksaan dilakukan dengan memastikan bahwa batas atas paket `bottom` pada sumbu z sama dengan batas bawah paket `top`. Selain itu, kedua paket harus memiliki irisan pada bidang horizontal, yaitu sumbu x dan y. Apabila seluruh kondisi terpenuhi, fungsi mengembalikan nilai `True` yang menunjukkan bahwa paket `bottom` dapat menopang paket `top`. Pemeriksaan ini memungkinkan penopangan sebagian selama terdapat irisan bidang horizontal antara kedua paket.

2.1.3 Pemeriksaan *Constraints*

Pemeriksaan *constraints* dilakukan untuk memastikan setiap *state* memenuhi batasan permasalahan. Terdapat enam fungsi yang digunakan untuk memeriksa batas dimensi truk, tumpang tindih paket, penopangan paket, kapasitas truk, serta validitas *state* secara keseluruhan.

- **Fungsi `_is_package_within_bounds()`**

Fungsi `_is_package_within_bounds()` digunakan untuk memastikan seluruh paket berada di dalam batas dimensi truk. Pemeriksaan dilakukan dengan membandingkan koordinat minimum dan maksimum setiap paket terhadap dimensi truk. Fungsi mengembalikan `False` apabila terdapat paket yang melewati batas atau dimensi truk tidak valid.

Kode 2.6 Implementasi fungsi `_is_package_within_bounds()`

```
def _is_package_within_bounds(self, state: State) -> bool:
    """Untuk mengecek batas truk (ukuran, bukan berat)"""
    for truck_id, boxes in
self._get_boxes_per_truck(state).items():
        dimension = self.truck_map[truck_id].dimension
        if any(size <= 0 for size in (dimension.w, dimension.l,
dimension.h)):
            return False
        for box in boxes.values():
            if not (0 <= box[0] < box[3] <= dimension.w and 0 <=
box[1] < box[4] <= dimension.l and 0 <= box[2] < box[5] <=
dimension.h):
                return False
    return True
```

- **Fungsi `_is_packages_overlapping()`**

Fungsi `_is_packages_overlapping()` digunakan untuk memeriksa apakah terdapat paket yang saling bertumpang tindih dalam truk yang sama. Pemeriksaan dilakukan terhadap setiap pasangan paket menggunakan fungsi `_boxes_overlap()`. Fungsi mengembalikan `True` apabila ditemukan paket yang bertumpang tindih.

Kode 2.7 Implementasi fungsi `_is_packages_overlapping()`

```
def _is_packages_overlapping(self, state: State) -> bool:
    for boxes in self._get_boxes_per_truck(state).values():
        placed = list(boxes.values())
        for i in range(len(placed)):
            for j in range(i + 1, len(placed)):
                if self._boxes_overlap(placed[i], placed[j]):
                    return True
    return False
```

- **Fungsi `_is_package_floating()`**

Fungsi `_is_package_floating()` digunakan untuk mendeteksi paket yang melayang. Paket dianggap tidak melayang apabila berada di lantai truk atau memiliki setidaknya satu paket penopang yang memenuhi fungsi `_supports()`. Fungsi mengembalikan `True` apabila ditemukan paket yang tidak memiliki penopang.

Kode 2.8 Implementasi fungsi `_is_package_floating()`

```
def _is_package_floating(self, state: State) -> bool:
    for boxes in self._get_boxes_per_truck(state).values():
        for package_id, box in boxes.items():
            if box[2] == 0:
                continue
            if not any(
                other_id != package_id and
self._supports(other_box, box)
                for other_id, other_box in boxes.items()
            ):
                return True
    return False
```

- **Fungsi `_is_fragile_supporting_another()`**

Fungsi `_is_fragile_supporting_another()` digunakan untuk memastikan paket pecah belah (*fragile*) tidak menopang paket lain. Pemeriksaan dilakukan terhadap setiap paket *fragile* menggunakan fungsi

`_supports()`. Fungsi mengembalikan `True` apabila terdapat paket fragile yang menjadi penopang paket lain.

Kode 2.9 Implementasi fungsi `_is_fragile_supporting_another()`

```
def _is_fragile_supporting_another(self, state: State) -> bool:
    for boxes in self._get_boxes_per_truck(state).values():
        for package_id, box in boxes.items():
            if not self.package_map[package_id].isFragile:
                continue
            if any(
                other_id != package_id and self._supports(box,
other_box)
                for other_id, other_box in boxes.items()
            ):
                return True
    return False
```

- **Fungsi `_is_truck_overloaded()`**

Fungsi `_is_truck_overloaded()` digunakan untuk memeriksa apakah total berat paket dalam suatu truk melebihi kapasitas maksimum. Fungsi menghitung total berat paket pada setiap truk dan membandingkannya dengan atribut `maxCapacity`. Fungsi mengembalikan `True` apabila terdapat truk yang kelebihan muatan.

Kode 2.10 Implementasi fungsi `_is_truck_overloaded()`

```
def _is_truck_overloaded(self, state: State) -> bool:
    for truck in self.trucks:
        weight = sum(self.package_map[package_id].weight for
package_id in state.get_inside_package_ids(truck.id))
        if weight > truck.maxCapacity:
            return True
    return False
```

- **Fungsi `is_state_valid()`**

Fungsi `is_state_valid()` digunakan untuk memvalidasi suatu state secara keseluruhan. Fungsi terlebih dahulu memeriksa struktur data, kelengkapan paket, orientasi, serta koordinat penempatan. Selanjutnya, seluruh fungsi pemeriksaan *constraints* dipanggil untuk memastikan konfigurasi paket memenuhi batasan permasalahan. Fungsi mengembalikan `True` hanya apabila seluruh pemeriksaan berhasil.

Kode 2.11 Implementasi fungsi `is_state_valid()`

```
def is_state_valid(self, state: State) -> bool:
```

```
    if not isinstance(state, State):
        return False
    if set(state.placements) != set(self.package_map):
        return False

    for placement in state.placements.values():
        if not isinstance(placement, PackagePlacement):
            return False
        if not isinstance(placement.orientation, Orientation):
            return False
        if not placement.is_inside:
            if placement.coordinate is not None:
                return False
            continue
        if placement.truck_id not in self.truck_map:
            return False
        coordinate = placement.coordinate
        if coordinate is None:
            return False
        if any(type(value) is not int for value in (coordinate.x,
coordinate.y, coordinate.z)):
            return False

    return (
        self._is_package_within_bounds(state)
        and not self._is_packages_overlapping(state)
        and not self._is_package_floating(state)
        and not self._is_fragile_supporting_another(state)
        and not self._is_truck_overloaded(state)
    )
```

2.1.4 Perhitungan *Objective Function*

Perhitungan *objective function* dilakukan untuk mengevaluasi kualitas suatu *state* berdasarkan total nilai paket yang berhasil dimuat ke dalam truk.

Kode 2.12 Implementasi fungsi `calculate_objective_value()`

```
def calculate_objective_value(self, state: State) -> int:
    if not self.is_state_valid(state):
        raise ValueError("Cannot calculate objective for an
invalid state")
    return sum(
        self.package_map[package_id].value
        for package_id, placement in state.placements.items()
        if placement.is_inside
    )
```

Fungsi ini terlebih dahulu memvalidasi *state* menggunakan `is_state_valid()`. Apabila *state* tidak valid, fungsi menghasilkan pengecualian `ValueError`. Jika valid, fungsi menjumlahkan nilai seluruh paket

yang berada di dalam truk berdasarkan atribut `is_inside`. Nilai yang dihasilkan digunakan sebagai dasar evaluasi solusi oleh algoritma *local search*, dengan tujuan memaksimalkan total nilai paket yang diangkut.

2.1.5 Pembangkitan *Initial State*

Pembangkitan *initial state* dilakukan untuk menghasilkan konfigurasi awal penempatan paket secara acak yang memenuhi seluruh *constraints*. Proses ini melibatkan tiga fungsi, yaitu `_candidate_coordinates()`, `_find_feasible_placement()`, dan `generate_initial_state()`.

- **Fungsi `_candidate_coordinates()`**

Fungsi `_candidate_coordinates()` digunakan untuk membangkitkan kandidat koordinat penempatan paket berdasarkan dimensi truk dan orientasi paket. Kandidat diperoleh dari titik asal, posisi di sekitar paket lain, permukaan penopang yang tidak *fragile*, serta koordinat acak. Seluruh kandidat disaring berdasarkan batas dimensi truk, kemudian diacak dan dibatasi jumlahnya sesuai parameter `max_candidates`.

Kode 2.13 Implementasi fungsi `_candidate_coordinates()`

```
def _candidate_coordinates(self, state: State, package_id: str,
truck_id: str, orientation: Orientation, rng: random.Random,
max_candidates: int = 32) -> list[Coordinate]:
    """Coba posisi pada lantai, di sisi paket, dan di atas
support."""
    truck = self.truck_map[truck_id]
    dimensions =
orientation.get_oriented_dimensions(self.package_map[package_id].
dimension)
    max_x = truck.dimension.w - dimensions.w
    max_y = truck.dimension.l - dimensions.l
    max_z = truck.dimension.h - dimensions.h
    if min(max_x, max_y, max_z) < 0:
        return []

    candidates = {(0, 0, 0)}
    support_heights = [0]
    for other_id, placement in state.placements.items():
        if other_id == package_id or placement.truck_id !=
truck_id:
            continue
        x0, y0, z0, x1, y1, z1 = self._get_package_box(other_id,
placement)
        candidates.update(((x1, y0, 0), (x0, y1, 0), (x1, 0, 0),
(0, y1, 0), (x0, y0, 0), (x1, y0, z0), (x0, y1, z0)))
        if not self.package_map[other_id].isFragile:
```

```
            support_heights.append(z1)
            candidates.update(((x0, y0, z1), (x1 - dimensions.w,
y0, z1), (x0, y1 - dimensions.l, z1)))

    for _ in range(max_candidates):
        candidates.add((rng.randint(0, max_x), rng.randint(0,
max_y), rng.choice(support_heights)))

    valid = sorted((x, y, z) for x, y, z in candidates if 0 <= x
<= max_x and 0 <= y <= max_y and 0 <= z <= max_z)
    rng.shuffle(valid)
    return [Coordinate(x, y, z) for x, y, z in
valid[:max_candidates]]
```

- **Fungsi `_find_feasible_placement()`**

Fungsi `_find_feasible_placement()` digunakan untuk mencari penempatan paket yang valid berdasarkan kombinasi orientasi dan koordinat. Fungsi mengacak urutan orientasi, kemudian membangkitkan kandidat koordinat menggunakan `_candidate_coordinates()`. Setiap kandidat diuji melalui `_move_package()`, dan fungsi mengembalikan *state* pertama yang valid atau `None` apabila tidak ditemukan penempatan yang memungkinkan.

Kode 2.14 Implementasi fungsi `_find_feasible_placement()`

```
def _find_feasible_placement(self, state: State, package_id: str,
truck_id: str, orientations: list[Orientation], rng:
random.Random, max_positions: int = 32) -> State | None:
    """Kembalikan salinan state dengan paket ditempatkan, jika
memungkinkan"""
    choices = orientations.copy()
    rng.shuffle(choices)
    for orientation in choices:
        coordinates = self._candidate_coordinates(state,
package_id, truck_id, orientation, rng, max_positions)
        for coordinate in coordinates:
            candidate = self._move_package(state, package_id,
truck_id, coordinate, orientation)
            if candidate is not None:
                return candidate
    return None
```

- **Fungsi `generate_initial_state()`**

Fungsi `generate_initial_state()` digunakan untuk membangkitkan *initial state* secara acak. Proses dimulai dengan menempatkan seluruh paket di luar truk, kemudian mengacak urutan paket dan truk. Setiap paket memiliki peluang 20% untuk tetap berada di luar truk, sedangkan paket lainnya dicoba untuk ditempatkan menggunakan

`_find_feasible_placement()`. Pemilihan probabilitas 20% merupakan heuristik untuk memberikan variasi pada solusi awal sekaligus menyisakan paket di luar truk yang dapat dimanfaatkan dalam eksplorasi *neighbor state*. Dengan demikian, konfigurasi awal tidak selalu terbentuk dari upaya memasukkan seluruh paket ke dalam truk. Fungsi kemudian mengembalikan konfigurasi akhir yang telah dibentuk sebagai *initial state*.

Kode 2.15 Implementasi fungsi `generate_initial_state()`

```
def generate_initial_state(self, rng: random.Random | None =
None) -> State:
    """Buat random feasible state dengan mengisi truk satu paket
demi satu paket"""
    rng = rng if rng is not None else random.Random()
    state = State({package.id: PackagePlacement(None, None,
package.orientation) for package in self.packages})
    if not self.is_state_valid(state):
        raise ValueError("Cannot construct an initial state for
this problem")

    package_ids = list(self.package_map)
    rng.shuffle(package_ids)
    for package_id in package_ids:
        # Sisakan sebagian paket di luar untuk variasi solusi
awal.
        if rng.random() < 0.20:
            continue
        truck_ids = list(self.truck_map)
        rng.shuffle(truck_ids)
        for truck_id in truck_ids:
            candidate = self._find_feasible_placement(state,
package_id, truck_id, Orientation.all_orientations(), rng)
            if candidate is not None:
                state = candidate
                break
    return state
```

2.1.6 Pembangkitan *Neighbor State*

Pembangkitan *neighbor state* dilakukan untuk menghasilkan konfigurasi baru dari *state* saat ini melalui operasi pertukaran, pemindahan, atau rotasi paket.

- **Fungsi `_swap_packages()`**

Fungsi `_swap_packages()` digunakan untuk menukar posisi dua paket, baik antartruk maupun antara paket di dalam dan di luar truk, tanpa mengubah orientasi masing-masing. Pertukaran dilakukan pada salinan *state*, kemudian

hasilnya divalidasi menggunakan `is_state_valid()`. Fungsi mengembalikan `None` apabila pertukaran tidak menghasilkan *state* baru yang valid.

Kode 2.16 Implementasi fungsi `_swap_packages()`

```
def _swap_packages(self, state: State, first_id: str, second_id:
str) -> State | None:
    """Tukar lokasi dua paket tanpa menukar orientasi"""
    if first_id not in self.package_map or second_id not in
self.package_map:
        raise ValueError("Unknown package ID")
    if first_id == second_id:
        return None
    first = state.placements[first_id]
    second = state.placements[second_id]
    if not first.is_inside and not second.is_inside:
        return None
    candidate = state.copy()
    candidate.placements[first_id] =
PackagePlacement(second.truck_id, second.coordinate,
first.orientation)
    candidate.placements[second_id] =
PackagePlacement(first.truck_id, first.coordinate,
second.orientation)
    return candidate if candidate != state and
self.is_state_valid(candidate) else None
```

- **Fungsi `_move_package()`**

Fungsi `_move_package()` digunakan untuk memindahkan paket ke koordinat atau truk lain, serta memungkinkan paket dikeluarkan dari truk dan paket dimasukkan ke dalam truk. Fungsi membuat salinan *state* dengan memperbarui penempatan dan orientasi paket apabila diberikan. Hasil pemindahan dikembalikan jika menghasilkan *state* baru yang valid, atau `None` jika tidak valid.

Kode 2.17 Implementasi fungsi `_move_package()`

```
def _move_package(self, state: State, package_id: str, truck_id:
str | None, coordinate: Coordinate | None, orientation:
Orientation | None = None) -> State | None:
    """Pindahkan paket. (None, None) = paket dikeluarkan dari
truk"""
    if package_id not in self.package_map:
        raise ValueError("Unknown package ID")
    if (truck_id is None) != (coordinate is None):
        raise ValueError("truck_id and coordinate must both be
set or both None")
    if truck_id is not None and truck_id not in self.truck_map:
        raise ValueError("Unknown truck ID")
    previous = state.placements[package_id]
```

```
    new_orientation = orientation if orientation is not None else
previous.orientation
    candidate = state.copy()
    candidate.placements[package_id] = PackagePlacement(truck_id,
coordinate, new_orientation)
    return candidate if candidate != state and
self.is_state_valid(candidate) else None
```

- **Fungsi `_rotate_package()`**

Fungsi `_rotate_package()` digunakan untuk merotasi paket sebesar 90° terhadap sumbu x, y, atau z tanpa mengubah koordinatnya. Rotasi dilakukan menggunakan metode `rotate()` pada orientasi paket. Apabila rotasi tidak mengubah dimensi paket atau menghasilkan *state* yang tidak valid, fungsi mengembalikan `None`.

Kode 2.18 Implementasi fungsi `_rotate_package()`

```
def _rotate_package(self, state: State, package_id: str, axis:
str) -> State | None:
    """Putar paket 90 derajat di posisi yang sama, jika state
hasilnya valid"""
    if package_id not in self.package_map:
        raise ValueError("Unknown package ID")
    previous = state.placements[package_id]
    if not previous.is_inside:
        return None
    new_orientation = previous.orientation.rotate(axis)
    dimension = self.package_map[package_id].dimension
    old_dims =
previous.orientation.get_oriented_dimensions(dimension)
    new_dims = new_orientation.get_oriented_dimensions(dimension)
    if (old_dims.w, old_dims.l, old_dims.h) == (new_dims.w,
new_dims.l, new_dims.h):
        return None  # Rotasi tidak mengubah geometri paket.
    candidate = state.copy()
    candidate.placements[package_id] =
PackagePlacement(previous.truck_id, previous.coordinate,
new_orientation)
    return candidate if self.is_state_valid(candidate) else None
```

- **Fungsi `generate_next_random_state()`**

Fungsi `generate_next_random_state()` digunakan untuk membangkitkan satu *neighbor state* yang valid melalui pemilihan operasi *swap*, *move*, atau *rotate* secara acak. Proses dilakukan hingga ditemukan *state* baru yang valid atau mencapai batas percobaan `max_attempts`, dengan nilai bawaan 80. Apabila tidak ditemukan *neighbor state* yang valid, fungsi mengembalikan `None`.

Kode 2.19 Implementasi fungsi `generate_next_random_state()`

```
def generate_next_random_state(self, current_state: State, rng:
random.Random | None = None, max_attempts: int = 80) -> State |
None:
    """Sampel satu neighbor valid melalui swap, move, atau
rotate. None = tidak menemukan neighbor valid"""
    if not self.is_state_valid(current_state):
        raise ValueError("Cannot generate a neighbor from an
invalid state")
    if max_attempts <= 0:
        raise ValueError("max_attempts must be positive")
    rng = rng if rng is not None else random.Random()
    inside_ids = [pid for pid, p in
current_state.placements.items() if p.is_inside]
    package_ids = list(self.package_map)

    for _ in range(max_attempts):
        operation = rng.choice(("swap", "move", "rotate"))

        if operation == "swap":
            if not inside_ids or len(package_ids) < 2:
                continue
            first_id = rng.choice(inside_ids)
            second_id = rng.choice([pid for pid in package_ids if
pid != first_id])
            candidate = self._swap_packages(current_state,
first_id, second_id)

        elif operation == "move":
            if not package_ids:
                continue
            package_id = rng.choice(package_ids)
            placement = current_state.placements[package_id]
            if placement.is_inside and rng.random() < 0.25:
                candidate = self._move_package(current_state,
package_id, None, None)
            elif self.truck_map:
                truck_id = rng.choice(list(self.truck_map))
                candidate =
self._find_feasible_placement(current_state, package_id,
truck_id, [placement.orientation], rng)
            else:
                candidate = None

        else:  # rotate
            if not inside_ids:
                continue
            candidate = self._rotate_package(current_state,
rng.choice(inside_ids), rng.choice(("x", "y", "z")))

        if candidate is not None:
            return candidate
    return None
```

- **Fungsi `generate_best_neighbor_state()`**

Fungsi `generate_best_neighbor_state()` digunakan untuk mencari *neighbor state* dengan nilai *objective function* tertinggi melalui tiga tahap, yaitu *swap*, *move*, dan *rotate*. Tahap *swap* mencoba seluruh pasangan paket, tahap *move* mencoba pemindahan setiap paket hingga maksimal 80 percobaan, sedangkan tahap *rotate* mencoba rotasi setiap paket di dalam truk terhadap ketiga sumbu. Setiap kandidat dibandingkan menggunakan `_better_state()` untuk memperoleh *state* terbaik yang ditemukan. Karena pemindahan dilakukan secara acak, fungsi ini tidak menjamin seluruh kemungkinan *neighbor* telah diperiksa.

Kode 2.20 Implementasi fungsi `generate_best_neighbor_state()`

```
def generate_best_neighbor_state(self, current_state: State, rng:
random.Random | None = None) -> State | None:
    """Hasilkan neighbor terbaik"""
    if not self.is_state_valid(current_state):
        raise ValueError("Cannot generate a neighbor from an
invalid state")
    rng = rng if rng is not None else random.Random()
    inside_ids = [pid for pid, p in
current_state.placements.items() if p.is_inside]
    package_ids = list(self.package_map)

    best_neighbour_state = current_state

    # Fase swap
    if inside_ids and len(package_ids) >= 2:
        for first_id in package_ids:
            for second_id in package_ids:
                if first_id == second_id:
                    continue
                candidate = self._swap_packages(current_state,
first_id, second_id)
                best_neighbour_state =
self._better_state(best_neighbour_state, candidate)

    # Fase move
    if package_ids:
        for id in package_ids:
            placement = current_state.placements[id]
            candidate = None
            max_move_iter = 80
            iter = 0
            while (not candidate and iter < max_move_iter):
                if placement.is_inside and rng.random() < 0.25:
                    candidate = self._move_package(current_state,
id, None, None)
                elif self.truck_map:
                    truck_id = rng.choice(list(self.truck_map))
                    candidate =
self._find_feasible_placement(current_state, id, truck_id,
[placement.orientation], rng)
```

```
                else:
                    candidate = None
                iter = iter + 1
            best_neighbour_state =
self._better_state(best_neighbour_state, candidate)

    # Fase rotate
    if inside_ids:
        for id in inside_ids:
            candidate = self._rotate_package(current_state, id,
"x")
            best_neighbour_state =
self._better_state(best_neighbour_state, candidate)

            candidate = self._rotate_package(current_state, id,
"y")
            best_neighbour_state =
self._better_state(best_neighbour_state, candidate)

            candidate = self._rotate_package(current_state, id,
"z")
            best_neighbour_state =
self._better_state(best_neighbour_state, candidate)

    return best_neighbour_state
```

2.2 Implementasi Algoritma *Hill Climbing*

Algoritma *Hill Climbing* mengimplementasikan berbagai variannya.

2.2.1 *Steepest Ascent Hill Climbing*

2.2.1.1 Inisialisasi Algoritma

Inisialisasi algoritma dilakukan tanpa penambahan parameter. Metode `__init__()` digunakan untuk menginisialisasi kelas `SteepestHC` sebagai kelas untuk algoritma ini.

Kode 2.21 Implementasi metode `__init__()`

```
def __init__(self, problem : Problem):
        super().__init__(problem)
```

2.2.1.2 Algoritma Utama

Algoritma utama untuk Steepest Ascent Hill Climbing diimplementasikan pada kelas `SteepestHC` metode `execute()`. Algoritma ini dimulai dengan pembuatan initial state (`generate_initial_state()`), lalu terus

membangkitkan suksesor terbaik (`generate_best_neighbor_state()`) yang memiliki nilai lebih besar daripada nilai maksimal saat ini pada loop while yang tertera. Setiap suksesor terbaik dicatat di `result.objective_values_history_per_iteration`. Ketika mencapai syarat terminasi, yaitu suksesor terbaik memiliki nilai yang tidak lebih besar daripada nilai maksimal saat ini (`is_terminate`), dilakukan exit dari loop. Disimpan juga atribut result seperti jumlah iterasi, durasi, best state/final state.

Kode 2.22 Implementasi metode `execute()`

```
def execute(self) -> SteepestResult:
        result = SteepestResult()
        current_state = self.problem.generate_initial_state()
        result.initial_state = current_state.copy()
        max_obj_value =
self.problem.calculate_objective_value(current_state)
        best_state = current_state

        iteration = 0
        start = time.time()

        result.final_objective_value_result = -1

result.objective_values_history_per_iteration.append(max_obj_valu
e)
        is_terminate = False

        while(True):
            current_state =
self.problem.generate_best_neighbor_state(best_state)
            current_objective_value =
self.problem.calculate_objective_value(current_state)


result.objective_values_history_per_iteration.append(current_obje
ctive_value)
            iteration += 1

            is_terminate =
(current_objective_value<=max_obj_value)

            if (is_terminate):
                end = time.time()
                result.duration_seconds = (end - start)
                result.final_state = best_state
                result.num_of_iterations = iteration
                result.final_objective_value_result =
max_obj_value
                return result
            else:
                best_state = current_state
                max_obj_value = current_objective_value
```

2.2.2 Sideways Hill Climbing

2.1.2.1 Inisialisasi Algoritma

Inisialisasi algoritma dilakukan dengan penambahan parameter max_side_move yang diinisialisasi sebagai 10 jika tidak dispesifikkan ketika pemanggilan algoritma. Metode `__init__()` digunakan untuk menginisialisasi kelas `SidewaysHC` sebagai kelas untuk algoritma ini.

Kode 2.23 Implementasi metode `__init__()`

```
def __init__(self, problem : Problem, max_side_move : int = 5):
        super().__init__(problem)
        self.max_side_move = max_side_move
```

2.1.2.2 Algoritma Utama

Algoritma utama untuk Sideways Hill Climbing diimplementasikan pada kelas `SidewaysHC` metode `execute()`. Algoritma ini dimulai dengan pembuatan initial state (`generate_initial_state()`), lalu terus membangkitkan suksesor terbaik (`generate_best_neighbor_state()`) yang memiliki nilai lebih besar atau sama dengan daripada nilai maksimal saat ini pada loop while yang tertera. Perlu dicatat bahwa pembangkitan suksesor dengan nilai yang sama dilakukan secara terbatas, maksimalnya sebanyak self.max_side_move. Setiap suksesor terbaik dicatat di `result.objective_values_history_per_iteration`. Ketika mencapai syarat terminasi, yaitu suksesor terbaik memiliki nilai yang tidak lebih besar daripada nilai maksimal saat ini atau ada suksesor dengan nilai sama tetapi banyak side_move sudah mencapai maksimal (`is_terminate`), dilakukan exit dari loop. Disimpan juga atribut result seperti jumlah iterasi, durasi, best state/final state.

Kode 2.24 Implementasi metode `execute()`

```
def execute(self) -> SidewaysResult:
        result = SidewaysResult()
        current_state = self.problem.generate_initial_state()
        result.initial_state = current_state.copy()
        max_obj_value =
self.problem.calculate_objective_value(current_state)
        best_state = current_state

        iteration = 0
        start = time.time()

        result.final_objective_value_result = -1
```

```
result.objective_values_history_per_iteration.append(max_obj_valu
e)
        is_terminate = False

        side_move = 0

        while(True):
            current_state =
self.problem.generate_best_neighbor_state(best_state)
            current_objective_value =
self.problem.calculate_objective_value(current_state)


result.objective_values_history_per_iteration.append(current_obje
ctive_value)
            iteration += 1

            if current_objective_value == max_obj_value:
                side_move += 1
                is_terminate =
(current_objective_value<max_obj_value) or
(current_objective_value == max_obj_value and side_move <=
self.max_side_move)

            if (is_terminate):
                end = time.time()
                result.duration_seconds = (end - start)
                result.final_state = best_state
                result.num_of_iterations = iteration
                result.final_objective_value_result =
max_obj_value
                return result
            else:
                best_state = current_state
                max_obj_value = current_objective_value
```

2.2.3 Stochastic Hill Climbing

2.1.2.3 Inisialisasi Algoritma

Inisialisasi algoritma dilakukan tanpa penambahan parameter. Metode `__init__()` digunakan untuk menginisialisasi kelas `StochasticHC` sebagai kelas untuk algoritma ini.

Kode 2.25 Implementasi metode `__init__()`

```
def __init__(self, problem : Problem):
        super().__init__(problem)
```

2.1.2.4 Algoritma Utama

Algoritma utama untuk Stochastic Hill Climbing diimplementasikan pada kelas `StochasticHC` metode `execute()`. Algoritma ini dimulai dengan pembuatan initial state (`generate_initial_state()`), lalu terus membangkitkan suksesor acak (`generate_next_random_state()`) dengan nilai lebih besar daripada nilai maksimal saat ini pada loop while yang tertera. Setiap suksesor yang dibangkitkan dicatat di `result.objective_values_history_per_iteration`. Ketika mencapai syarat terminasi, yaitu suksesor terbaik memiliki nilai yang tidak lebih besar daripada nilai maksimal saat ini (`is_terminate`), dilakukan exit dari loop. Disimpan juga atribut result seperti jumlah iterasi, durasi, best state/final state.

Kode 2.26 Implementasi metode `execute()`

```
def execute(self) -> StochasticResult:
        result = StochasticResult()
        current_state = self.problem.generate_initial_state()
        result.initial_state = current_state.copy()
        max_obj_value =
self.problem.calculate_objective_value(current_state)
        best_state = current_state

        iteration = 0
        start = time.time()

        result.final_objective_value_result = -1

result.objective_values_history_per_iteration.append(max_obj_valu
e)
        is_terminate = False

        while(True):
            current_state =
self.problem.generate_next_random_state(best_state)
            current_objective_value =
self.problem.calculate_objective_value(current_state)


result.objective_values_history_per_iteration.append(current_obje
ctive_value)
            iteration += 1

            is_terminate =
(current_objective_value<=max_obj_value)

            if (is_terminate):
                end = time.time()
                result.duration_seconds = (end - start)
                result.final_state = best_state
                result.num_of_iterations = iteration
                result.final_objective_value_result =
```

```
max_obj_value
                return result
            else:
                best_state = current_state
                max_obj_value = current_objective_value
```

2.2.4 Random Restart Hill Climbing

2.2.4.1. Inisialisasi Algoritma

Inisialisasi algoritma dilakukan dengan penambahan parameter max_restart, yaitu banyak restart maksimal yang boleh dilakukan. Metode `__init__()` digunakan untuk menginisialisasi kelas `RandomRestartHC` sebagai kelas untuk algoritma ini.

Kode 2.27 Implementasi metode `__init__()`

```
def __init__(self, problem : Problem,
                    max_restart : int = 10):
        super().__init__(problem)
        self.max_restart = max_restart
```

2.2.4.2. Algoritma Utama

Algoritma utama untuk Stochastic Hill Climbing diimplementasikan pada kelas `StochasticHC` metode `execute()`. Algoritma ini dimulai dengan pembuatan initial state (`generate_initial_state()`), lalu terus membangkitkan suksesor acak (`generate_next_random_state()`) dengan nilai lebih besar daripada nilai maksimal saat ini pada loop while yang tertera. Setiap suksesor yang dibangkitkan dicatat di `result.objective_values_history_per_iteration`. Ketika mencapai syarat terminasi, yaitu suksesor terbaik memiliki nilai yang tidak lebih besar daripada nilai maksimal saat ini (`is_terminate`), dilakukan exit dari loop, atau dapat dilakukan restart.

Restart berarti state awal diinisialisasi ulang secara random, dan dicari lagi suksesor acak yang lebih baik hingga is_terminate menjadi true. Restart dibatasi sebanyak max_restart. Setiap restart hendak dilakukan, banyak iterasinya disimpan di dalam iteration_per_restart. Disimpan juga atribut result seperti jumlah iterasi, durasi, best state/final state.

Kode 2.28 Implementasi metode `execute()`

```
def execute(self) -> RandomRestartResult:
        result = RandomRestartResult()
```

```
        current_state = self.problem.generate_initial_state()
        max_obj_value =
self.problem.calculate_objective_value(current_state)
        global_best_state = current_state.copy()
        result.initial_state = current_state.copy()

        iteration = 0
        start = time.time()

        result.final_objective_value_result = -1

result.objective_values_history_per_iteration.append(max_obj_valu
e)
        num_of_restarts = 0
        is_restart = False

        local_best_state = current_state.copy()
        local_max_obj_value =
self.problem.calculate_objective_value(current_state)

        while(True):
            current_state =
self.problem.generate_next_random_state(local_best_state)
            current_objective_value =
self.problem.calculate_objective_value(current_state)


result.objective_values_history_per_iteration.append(current_obje
ctive_value)
            iteration += 1

            if (current_objective_value <= local_max_obj_value):
                is_restart = True
            else:
                local_best_state = current_state.copy()
                local_max_obj_value = current_objective_value


            if (is_restart and num_of_restarts <=
self.max_restart):
                num_of_restarts += 1
                result.iterations_per_restarts.append(iteration)
                if local_max_obj_value > max_obj_value:
                    max_obj_value = local_max_obj_value
                    global_best_state = local_best_state.copy()

                result.iterations_per_restarts.append(iteration)
                iteration = 0

                current_state =
self.problem.generate_initial_state()
                result.initial_state = current_state
                local_max_obj_value =
self.problem.calculate_objective_value(current_state)
                local_best_state = current_state.copy()
            elif (is_restart and num_of_restarts >
```

```
self.max_restart):
                end = time.time()
                result.duration_seconds = (end - start)
                result.final_state = global_best_state
                result.final_objective_value_result =
max_obj_value
                return result
```

2.2.5 Hasil dan Analisis Eksperimen

Eksperimen *Hill Climbing* dilakukan menggunakan berkas `untuk_laporan.json` yang memuat 3 truk dan 15 paket. Pengujian mencakup empat varian algoritma, yaitu *Steepest Ascent*, *Sideways*, *Stochastic*, dan *Random Restart Hill Climbing*. Setiap varian dijalankan sebanyak tiga kali untuk mengamati kualitas solusi, jumlah iterasi, durasi pencarian, serta konsistensi hasil.

2.2.5.1 Hasil Eksperimen

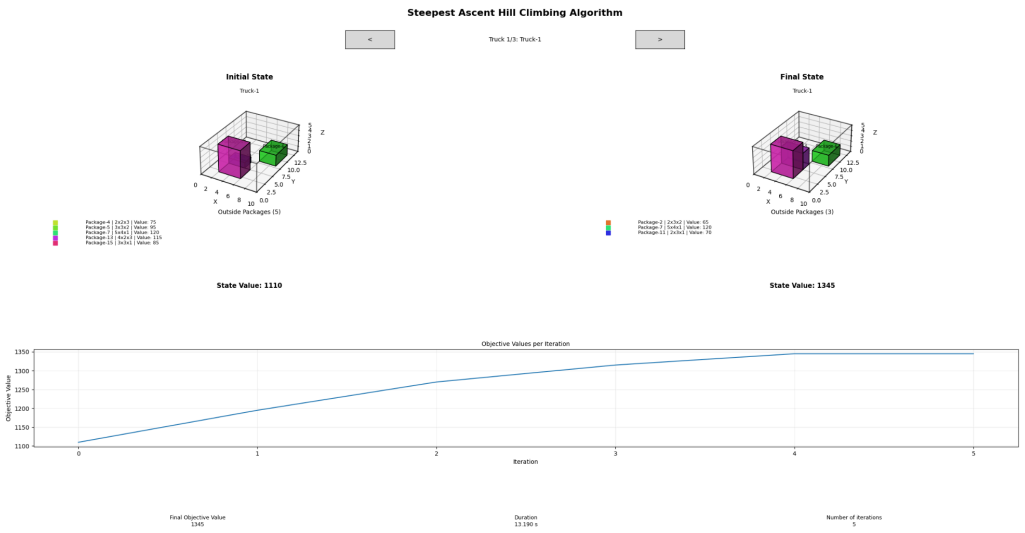
Tabel 2.2.1 Hasil eksperimen algoritma *Hill Climbing*

Algoritma	Run	Objective Awal	Objective Akhir	Iterasi	Durasi (s)
Steepest Ascent	1	1140	1320	5	12,367
	2	1110	1345	5	13,190
	3	1240	1320	3	7,941
Sideways	1	1200	1300	5	18,098
	2	1220	1310	4	9,744
	3	885	1335	5	6,802
Stochastic	1	1215	1215	1	0,007
	2	1230	1230	1	0,001
	3	1225	1225	1	0,011
Random Restart	1	1195	1290	-	2,211
	2	1235	1310	-	2,853
	3	1245	1255	-	2,391

Untuk mempermudah perbandingan, rata-rata hasil ketiga percobaan disajikan pada Tabel 2.2.2.

Tabel 2.2.2 Rekapitulasi hasil eksperimen *Hill Climbing*

Algoritma	Rata-rata Objective Akhir	Rata-rata Durasi (s)
Steepest Ascent	1328,33	11,166
Sideways	1315,00	11,548
Stochastic	1223,33	0,006
Random Restart	1285,00	2,485



Gambar 2.2.1 Visualisasi hasil Steepest Ascent Hill Climbing pada percobaan kedua

2.2.5.2 Analisis Hasil Eksperimen

Berdasarkan Tabel 2.2.2, *Steepest Ascent Hill Climbing* menghasilkan rata-rata nilai *objective function* tertinggi, yaitu 1328,33, diikuti oleh *Sideways Hill Climbing* sebesar 1315,00. Kedua varian tersebut menunjukkan peningkatan nilai objektif dalam seluruh percobaan. Hal ini berkaitan dengan pembangkitan dan pemilihan kandidat dengan nilai objektif lebih tinggi pada setiap iterasi.

Sebaliknya, *Stochastic Hill Climbing* tidak menunjukkan peningkatan nilai objektif pada ketiga percobaan. Algoritma berhenti setelah satu iterasi karena kandidat acak pertama tidak memberikan peningkatan. Kondisi ini menunjukkan

bahwa penghentian pencarian setelah satu kandidat gagal dapat membatasi eksplorasi solusi.

Dari segi durasi, *Stochastic Hill Climbing* memiliki rata-rata waktu pencarian terendah, yaitu 0,006 detik, tetapi tidak menghasilkan peningkatan kualitas solusi. Sementara itu, *Steepest Ascent* dan *Sideways* membutuhkan waktu lebih lama karena melakukan pemeriksaan lebih banyak kandidat.

Dari segi konsistensi, *Steepest Ascent* menghasilkan nilai akhir antara 1320–1345, sedangkan *Sideways* menghasilkan nilai antara 1300–1335. Keduanya menunjukkan hasil yang relatif konsisten. *Stochastic* menghasilkan rentang lebih kecil, yaitu 1215–1230, tetapi seluruh percobaan berhenti tanpa peningkatan nilai objektif.

Hasil sementara *Random Restart* menunjukkan nilai akhir antara 1255–1310. Namun, evaluasi terhadap efektivitas mekanisme *restart* masih memerlukan perbaikan pencatatan jumlah *restart* dan iterasi per *restart*.

Secara keseluruhan, *Steepest Ascent Hill Climbing* memberikan rata-rata kualitas solusi tertinggi pada eksperimen ini. Sementara itu, durasi pencarian yang lebih rendah tidak selalu menunjukkan kinerja lebih baik karena perlu dipertimbangkan bersama kualitas solusi yang dihasilkan.

2.3 Implementasi Algoritma *Simulated Annealing*

Algoritma Simulated Annealing diimplementasikan untuk mencari konfigurasi penempatan paket yang optimal melalui proses eksplorasi state dengan mempertimbangkan penurunan temperatur. Setiap state merepresentasikan suatu konfigurasi penempatan paket, sedangkan nilai **E** (objective function) dihitung berdasarkan total nilai paket yang berhasil dimuat ke dalam truk. Proses pencarian dilakukan melalui tahapan inisialisasi state awal, pembangkitan state tetangga (neighbor), evaluasi objective function, dan penerimaan state baru berdasarkan probabilitas secara, penurunan suhu berulang hingga suhu mencapai batas minimum yang ditentukan atau mencapai batas maksimum dari iterasi. Implementasi algoritma ini dijelaskan mulai dari kelas-kelas bersangkutan yang belum dijelaskan hingga fungsi-fungsi yang berkaitan, terutama dalam algoritma simulated annealing yang diimplementasikan. melalui fungsi-fungsi yang terdapat pada kelas SimulatedAnnealing, dilanjutkan dengan hasil dan analisis eksperimen terhadap variasi suhu awal (initial temperature) dan laju pendinginan (cooling rate).

2.2.1 Class SimulatedAnnealing

Kelas ini merupakan kelas yang akan menjalankan algoritma simulated annealing. Kelas ini merupakan turunan dari kelas LocalSearch agar polimorfisme dapat dimanfaatkan di bagian main program.

Kode 2.29 Pemanggilan method execute LocalSearch yang mengakses execute dari kelas SimulatedAnnealing yang memanfaatkan konsep polimorfisme

```
    def run(self):
        try:
            problem = Reader(self.inputFile).load_problem()
        except FileNotFoundError:
            print(f"File {self.inputFile} doesn't exist")
            print("Put your file inside the tests directory")
            return

        try:
            localsearch =
LocalSearchAlgorithmFactory().getLocalSearchAlgorithm(self.algori
thm, problem, **self.kwargs)
        except ValueError:
            print(f"Selected algorithm {self.algorithm} doesn't
exist")
            return

        result = localsearch.execute()
        result.visualize(problem, self.outputFile, 15, 12)
```

2.2.2 Class SimulatedAnnealingResult

Kelas ini merupakan kelas yang akan membungkus nilai-nilai yang perlu ditampilkan ketika algoritma simulated annealing selesai dijalankan. Kelas ini mewarisi kelas LocalSearchResult agar atribut-atribut yang dimiliki (wajib ditampilkan bagi semua algoritma local search) dapat dimiliki oleh kelas ini juga. Selain itu, konsep *inheritance* membantu agar visualisasi dapat memanfaatkan konsep polimorfisme.

Kode 2.30 Implementasi kode LocalSearchResult

```
@dataclass
class SimulatedAnnealingResult(LocalSearchResult):
    algorithm_name : str = "Simulated Annealing"
    acceptance_history_per_iteration: list[float] =
field(default_factory=list)
    stuck_frequency : float = 0.0

    def _get_charts(self) -> list[ChartConfig]:
```

```
        return [
            ChartConfig(
                "Objective Values per Iteration", "Iteration",
"Objective Value",
                [ChartSeries("Objective Value",
self.objective_values_history_per_iteration)],
            ),
            ChartConfig(
                "Acceptance Probability per Iteration",
"Iteration", "Probability",
                [ChartSeries("Acceptance Probability",
self.acceptance_history_per_iteration)],
            ),
        ]

    def _get_extra_metrics(self) -> list[MetricConfig]:
        return [MetricConfig("Stuck Frequency",
self.stuck_frequency)]
```

Kode 2.31 Pemanggilan method visualize dari kelas LocalSearchResult yang memanfaatkan data dari SimulatedAnnealing dengan memanfaatkan konsep polimorfisme

```
def run(self):
        try:
            problem = Reader(self.inputFile).load_problem()
        except FileNotFoundError:
            print(f"File {self.inputFile} doesn't exist")
            print("Put your file inside the tests directory")
            return

        try:
            localsearch =
LocalSearchAlgorithmFactory().getLocalSearchAlgorithm(self.algori
thm, problem, **self.kwargs)
        except ValueError:
            print(f"Selected algorithm {self.algorithm} doesn't
exist")
            return

        result = localsearch.execute()
        result.visualize(problem, self.outputFile, 15, 12)
```

2.2.3 Method __init__ pada Simulated Annealing

Method ini digunakan untuk mengkonstruksi kelas Simulated Annealing, di mana problem merupakan representasi dari masalah yang diselesaikan, temperature merupakan temperatur awal dari Simulated Annealing yang akan dijalankan, min_temperature berarti suhu minimal agar algoritma ini tetap berjalan, alpha yang berarti seberapa cepat penurunan suhu akan dilakukan,

dan max_iteration yang berarti batas maksimum iterasi dalam menjalankan algoritma ini.

Kode 2.32 Implementasi kode __init__ pada Simulated Annealing

```
    def __init__(self, problem : Problem,
                 temperature : float = 100,
                 min_temperature : float = 0.1,
                 alpha : float = 0.95,
                 max_iteration : int = 100):
        super().__init__(problem)

        self.temperature = temperature
        self.min_temperature = min_temperature
        self.max_iteration = max_iteration
        self.alpha = alpha
```

2.2.4 Method execute pada SimulatedAnnealing

Method ini merupakan method utama untuk menjalankan algoritma Simulated Annealing.

2.2.4.1 Inisialisasi data dan state

Bagian ini menginisialisasi data-data awal yang diperlukan untuk membantu proses algoritma Simulated Annealing berjalan serta mempersiapkan output berupa SimulatedAnnealingResult yang akan dihasilkan dan akan ditampilkan nantinya. Di bagian sini juga dihasilkan state awal dengan memanfaatkan method generate_initial_state() yang ada di kelas problem yang telah dijelaskan sebelumnya.

Kode 2.33 Inisialisasi Data dan State

```
    def execute(self) -> SimulatedAnnealingResult:
        result = SimulatedAnnealingResult()

        current_state = self.problem.generate_initial_state()
        current_objective_value =
self.problem.calculate_objective_value(current_state)

        result.initial_state = current_state.copy()

        result.final_state = current_state.copy()
        result.final_objective_value_result =
current_objective_value

        iteration = 0
        stuck_count = 0
        start = time.time()
```

2.2.4.2 Membangkitkan neighbor

Bagian ini sudah memasuki loop yang terus berjalan selama kondisi running. Di bagian ini juga akan dihasilkan neighbor dengan yang generate_next_random_state(current_state) yang ada di kelas problem yang telah dijelaskan sebelumnya.

Kode 2.34 Membangkitkan neighbor

```
          while(self._is_running(iteration)):
            next_state =
self.problem.generate_next_random_state(current_state)
            . . .
```

2.2.4.3 Evaluasi objective function

Bagian ini sudah memasuki loop yang terus berjalan selama kondisi running. Di bagian ini juga akan dilakukan evaluasi terhadap nilai objective function dari next state dan juga menghitung nilai deltanya sebelum menentukan next state.

Kode 2.35 Evaluasi Objective Function

```
          while(self._is_running(iteration)):
            . . .
            next_objective_value=
self.problem.calculate_objective_value(next_state)

            objective_value_difference = next_objective_value -
current_objective_value

            . . .
```

2.2.4.4 Menentukan next state

Bagian ini akan mengevaluasi hasil dari objective value difference (ΔE), di mana jika hasil dari ΔE lebih dari atau sama dengan nol, state saat ini akan berganti ke state neighbor yang dibangkitkan tersebut. Akan tetapi, jika kurang dari, maka akan menghitung nilai probability $e^{\Delta E/T}$, di mana jika nilai probabilitasnya lebih dari suatu nilai random maka state akan diterima sehingga current state akan menjadi next state. Jika tidak, perhitungan stuck akan bertambah

Kode 2.36 Menentukan next state

```
          while(self._is_running(iteration)):
            . . .
            acceptance_probability = 1.0
            if
```

```
self._is_move_to_better(objective_value_difference):
                current_state = next_state
                current_objective_value = next_objective_value
            else:
                acceptance_probability =
self._calculate_acceptance_probability(objective_value_difference
)
                if
self._is_worse_accepted(acceptance_probability):
                    current_state = next_state
                    current_objective_value =
next_objective_value
                else:
                    stuck_count += 1

            if
self._is_new_best_objective_value(result.final_objective_value_re
sult, current_objective_value):
                result.final_objective_value_result =
current_objective_value
                result.final_state = current_state.copy()


result.objective_values_history_per_iteration.append(current_obje
ctive_value)

result.acceptance_history_per_iteration.append(acceptance_probabi
lity)
            . . .
```

2.2.4.5 Cooling Algorithm

Bagian ini akan mengupdate temperature agar menurun. Sebenarnya terdapat sejumlah metode, seperti linear cooling, logarithmic cooling, ataupun exponential cooling. Dalam kasus ini akan memilih exponential cooling.

Kode 2.37 Bagian cooling algorithm

```
          while(self._is_running(iteration)):
            . . .

            iteration += 1
            self._exponential_cooling()
        . . .

        end = time.time()
        result.duration_seconds = (end - start)
        result.stuck_frequency = stuck_count / iteration if
iteration > 0 else 0.0
        return result
```

2.2.4.6 Pencatatan terakhir

Bagian ini akan mengisi sejumlah variabel terkait data yang akan divisualisasikan.

Kode 2.38 Pencatatan terakhir

```
        . . .

        end = time.time()
        result.duration_seconds = (end - start)
        result.stuck_frequency = stuck_count / iteration if
iteration > 0 else 0.0
        return result
```

2.2.5 Sejumlah Helper

Berikut ini sejumlah helper untuk membantu agar kode lebih *readable.* Kegunaan masing-masing helper sesuai dengan nama dari setiap method

Kode 2.39 Implementasi helper

```
    def _is_move_to_better(self, objective_value_difference :
float):
        return objective_value_difference >= 0

    def _is_running(self, iteration) -> bool:
        return self.temperature >= self.min_temperature and
iteration < self.max_iteration

    def _exponential_cooling(self):
        self.temperature *= self.alpha

    def _calculate_acceptance_probability(self,
objective_value_difference: float) -> float:
        return math.exp(objective_value_difference /
self.temperature)

    def _is_worse_accepted(self, acceptance_probability : float)
-> bool:
        return acceptance_probability > random()

    def _is_new_best_objective_value(self, best_objective_value :
float, new_objective_value : float) -> bool:
        return best_objective_value < new_objective_value
```

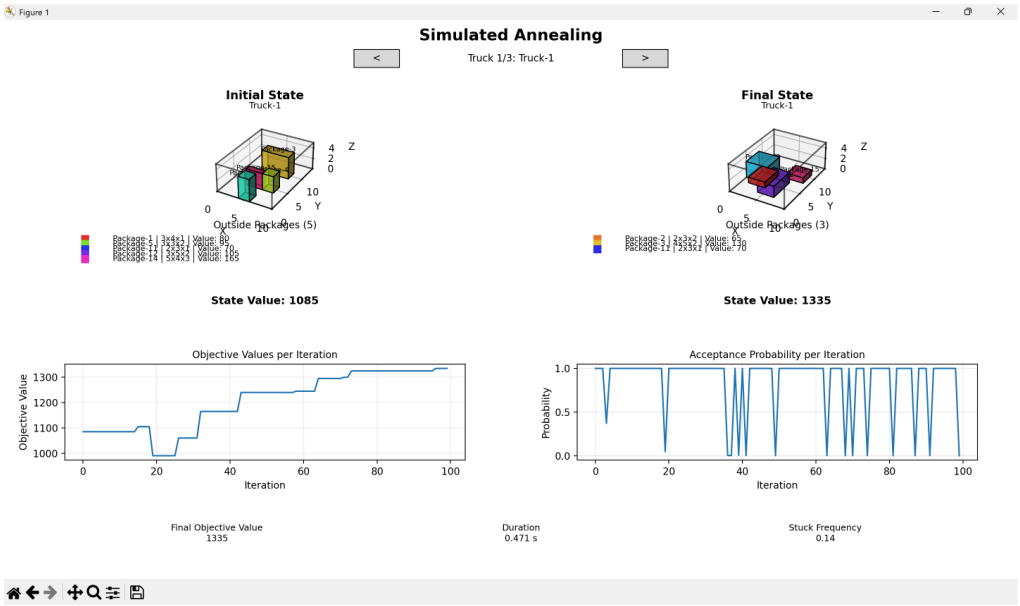
2.2.6 Hasil dan Analisis Eksperimen

2.2.5.1 Hasil

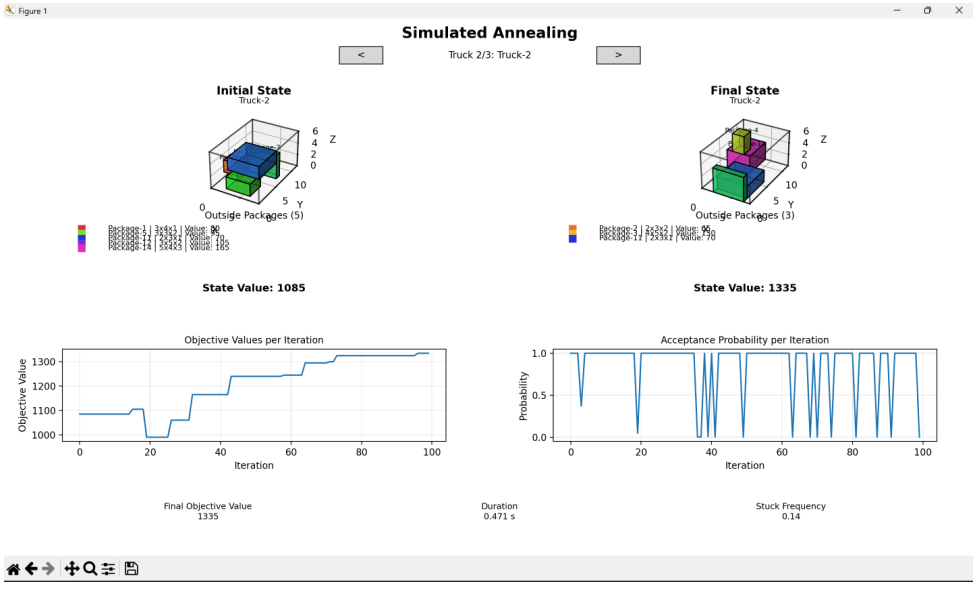
Berikut ini hasil dari sejumlah run.

Run 1

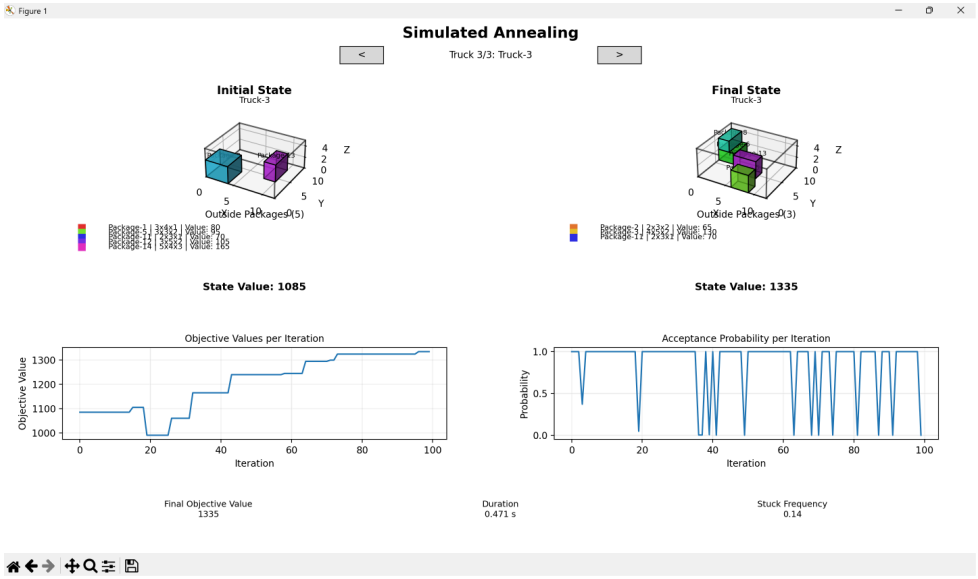
- python src/main.py simulated_annealing untuk_laporan.json SA_RUN_1.pickle --temperature 100 --alpha 0.95 --min_temperature 0.1 --max_iteration 100
- Truck 1



- Truck 2

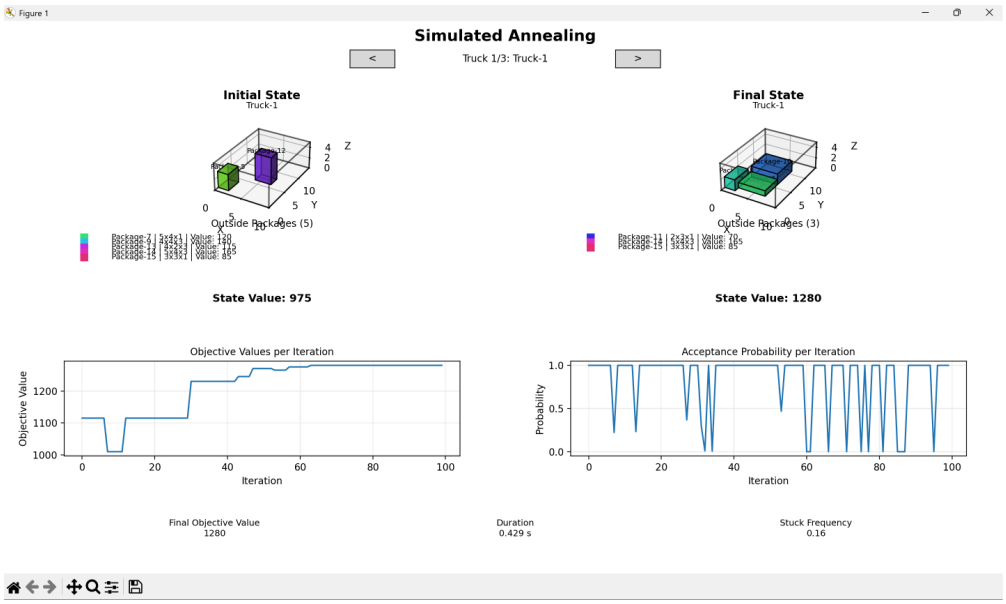


- Truck 3

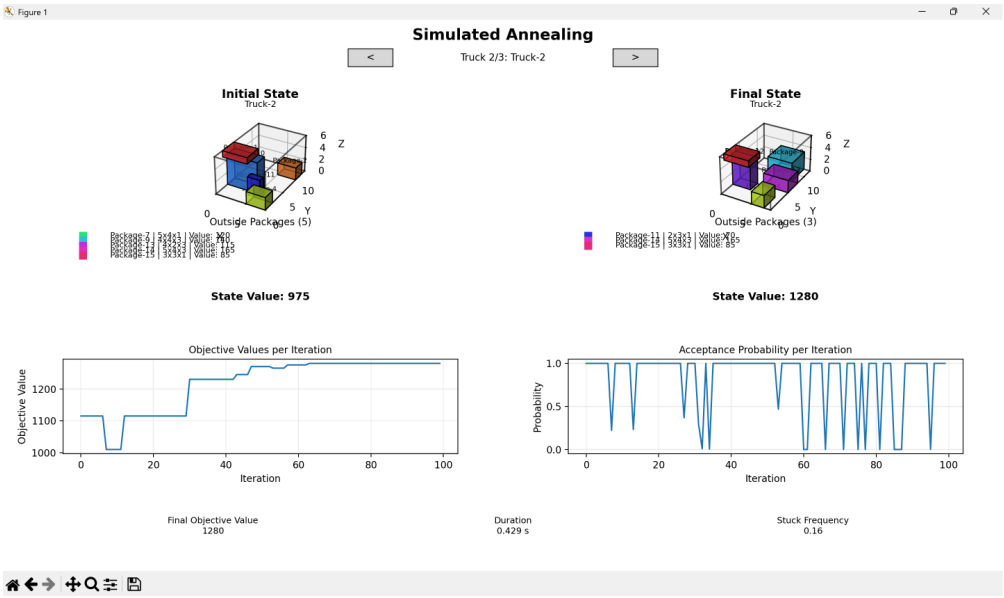


Run 2

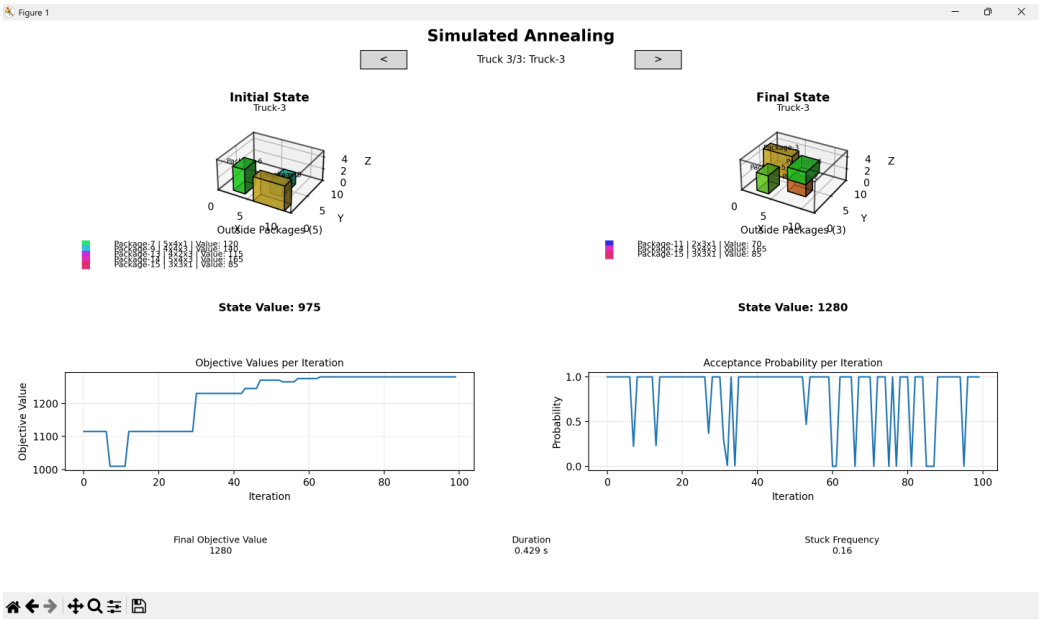
- python src/main.py simulated_annealing untuk_laporan.json SA_RUN_2.pickle --temperature 100 --alpha 0.95 --min_temperature 0.1 --max_iteration 100
- Truck 1



- Truck 2



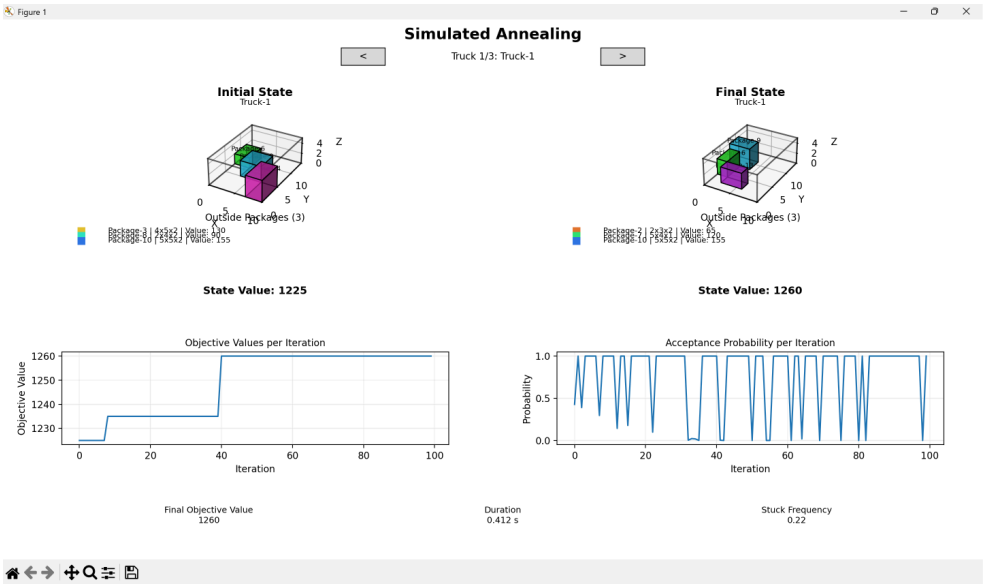
- Truck 3



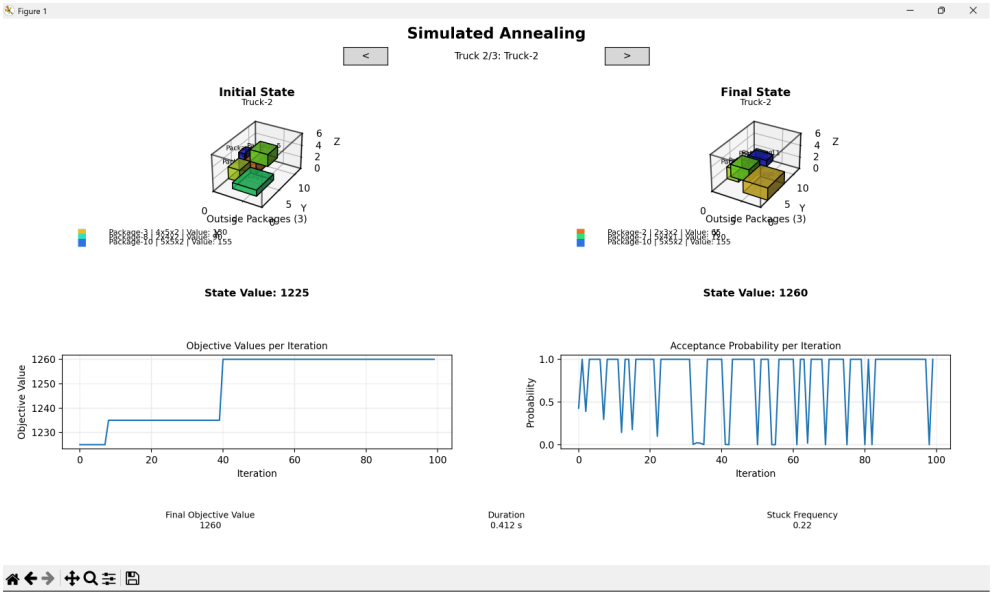
Run 3

- Truck 1

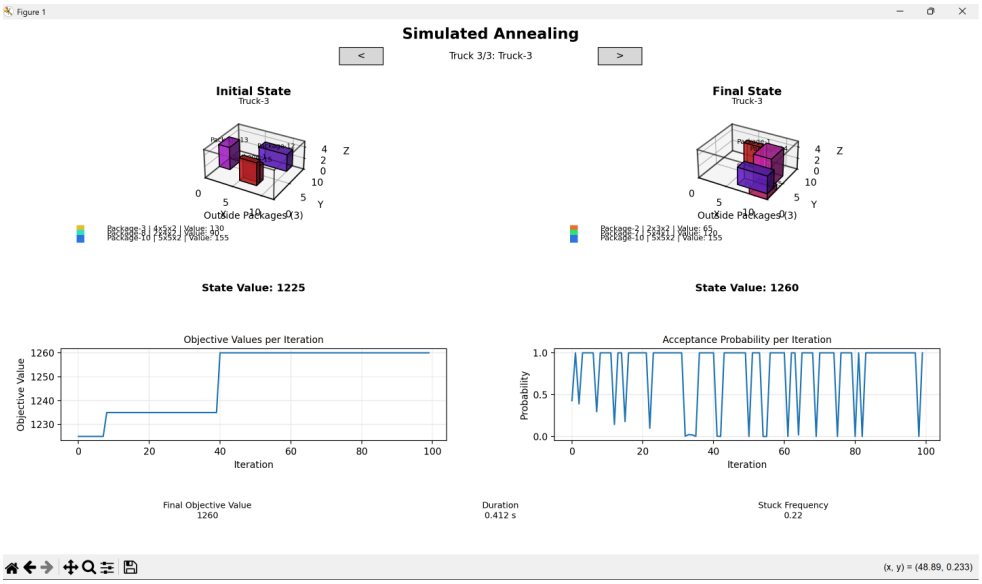
  python src/main.py simulated_annealing untuk_laporan.json SA_RUN_3.pickle --temperature 100 --alpha 0.95 --min_temperature 0.1 --max_iteration 100



- Truck 2



- Truck 3



2.2.5.2 Analisis Eksperimen

Tabel 2.2. 5 Hasil eksperimen Simulated Annealing

Parameter	**Eksperimen 1**	**Eksperimen 2**	**Eksperimen 3**
Alpha (Cooling Rate)	0.95	0.95	0.95
Temperature (Start Temperature)	100	100	100
Min Temperature	0.1	0.1	0.1
Max Iteration	100	100	100
Initial State Value	1085	975	1225
Final State Value	1335	1280	1280
Stuck Frequency	0.14	0.16	0.22
Durasi Eksekusi	0.471 s	0.429 s	0.412 s

Di hasil eksperimen 1, diperoleh final state yang lebih baik dibandingkan eksperimen 2 dan eksperimen 3. Di mana stuck frequency di eksperimen 1 juga lebih rendah sehingga dapat lebih mengeksplorasi banyak ruang, dibandingkan eksperimen 2 dan 3, bahkan di eksperimen 3 stuck cukup tinggi sehingga pada akhirnya eksperimen 2 yang memiliki initial state value lebih rendah pada akhirnya dapat memperoleh hasil yang serupa (hal

ini juga bisa dimungkinkan karena neighbor yang di-*generate* dengan *random*.

Pada masing-masing eksperimen yang dilakukan, terlihat bahwa Simulated Annealing memberikan *final* state(*best* state) yang lebih baik dibandingkan dengan *initial* statenya. Terlihat juga bahwa Simulated Annealing dapat mengeksplorasi, dalam hal ini pergi ke state yang nilai objective valuenya lebih rendah agar tidak stuck dan diharapkan dapat mencapai nilai objektif value yang lebih baik lagi. Algoritma ini juga menunjukkan bahwa next neighbor yang di-*generate* secara random mengakibatkan meskipun initial state valuenya berbeda, bisa pada akhirnya mendapatkan final/best state yang sama.

Dengan demikian hal ini semakin menegaskan bahwa dengan kemampuan Simulated Annealing untuk berpindah ke state yang object valuenya lebih rendah justru membantunya untuk tidak terjebak di lokal optima dan dapat memperoleh state yang lebih baik nantinya.

2.4 Implementasi Algoritma *Genetic Algorithm*

Algoritma *Genetic Algorithm* diimplementasikan untuk mencari konfigurasi penempatan paket yang optimal melalui proses evolusi populasi. Setiap individu merepresentasikan suatu *state* penempatan paket, sedangkan nilai *fitness* dihitung berdasarkan total nilai paket yang berhasil dimuat ke dalam truk. Proses pencarian dilakukan melalui tahapan inisialisasi populasi, seleksi induk (*selection*), persilangan (*crossover*), dan mutasi (*mutation*) secara berulang hingga mencapai jumlah iterasi yang ditentukan. Implementasi algoritma ini dijelaskan melalui fungsi-fungsi yang terdapat pada kelas `GeneticAlgorithm`, dilanjutkan dengan hasil dan analisis eksperimen terhadap variasi jumlah populasi dan iterasi.

2.4.1 Inisialisasi Algoritma dan Populasi

Inisialisasi algoritma dan populasi dilakukan untuk menentukan parameter pencarian serta membangkitkan kumpulan individu awal. Proses ini melibatkan dua metode, yaitu `__init__()` dan `_initialize_population()`.

- **Metode `__init__()`**

Metode `__init__()` digunakan untuk menginisialisasi parameter algoritma, yaitu `population_size` sebagai jumlah individu dalam populasi, `max_iterations` sebagai jumlah generasi yang akan dijalankan, dan `seed` untuk mengatur pembangkitan bilangan acak. Parameter `seed` digunakan untuk mengontrol pembangkitan bilangan acak agar hasil pencarian dapat direproduksi ketika menggunakan nilai *seed* yang sama. Dengan demikian, penggunaan *seed* memungkinkan eksperimen dilakukan secara konsisten untuk membandingkan pengaruh parameter algoritma. Metode ini juga melakukan validasi untuk memastikan jumlah populasi minimal 2 dan jumlah iterasi tidak negatif.

Kode 2.40 Implementasi metode `__init__()`

```
def __init__(self, problem: Problem, population_size: int = 20,
max_iterations: int = 100, seed: int | None = None):
    super().__init__(problem)
    if type(population_size) is not int or population_size < 2:
        raise ValueError("population_size must be an integer >=
2")
    if type(max_iterations) is not int or max_iterations < 0:
        raise ValueError("max_iterations must be a nonnegative
integer")
    self.population_size = population_size
    self.max_iterations = max_iterations
    self.seed = seed
```

- **Metode `_initialize_population()`**

Metode `_initialize_population()` digunakan untuk membangkitkan populasi awal sebanyak `population_size` individu. Setiap individu dibangkitkan secara acak melalui fungsi `generate_initial_state()` pada kelas `Problem`, sehingga seluruh individu yang dihasilkan memenuhi *constraints* permasalahan.

Kode 2.41 Implementasi metode `_initialize_population()`

```
def _initialize_population(self, rng: random.Random) ->
list[State]:
    return [self.problem.generate_initial_state(rng) for _ in
range(self.population_size)]
```

2.4.2 Perhitungan Fitness

Perhitungan *fitness* dilakukan untuk mengevaluasi kualitas setiap individu dalam populasi. Nilai *fitness* ditentukan berdasarkan *objective function* yang telah didefinisikan pada kelas `Problem`.

Metode `_fitness()` digunakan untuk menghitung nilai *fitness* suatu individu dengan memanggil fungsi `calculate_objective_value()`. Nilai yang dihasilkan merepresentasikan total nilai paket yang berhasil dimuat ke dalam truk dengan tetap memenuhi seluruh *constraints*. Semakin tinggi nilai *fitness*, semakin baik kualitas individu tersebut.

Kode 2.42 Implementasi metode `_fitness()`

```
def _fitness(self, individual: State) -> float:
    return self.problem.calculate_objective_value(individual)
```

2.4.3 Seleksi Induk

Seleksi induk dilakukan untuk memilih individu yang akan digunakan dalam proses persilangan (*crossover*). Proses ini menggunakan metode *roulette wheel selection*, yaitu pemilihan secara acak dengan probabilitas yang bergantung pada nilai *fitness* setiap individu.

Metode `_select_parent()` digunakan untuk memilih satu individu dari populasi berdasarkan bobot nilai *fitness*. Individu dengan nilai *fitness* lebih tinggi memiliki peluang lebih besar untuk terpilih sebagai induk. Apabila terdapat nilai *fitness* negatif, seluruh bobot disesuaikan dengan mengurangi nilai minimum. Jika total bobot tidak positif, pemilihan dilakukan secara acak dengan peluang yang sama untuk setiap individu.

Kode 2.43 Implementasi metode `_select_parent()`

```
def _select_parent(self, population: list[State], fitnesses:
list[float], rng: random.Random) -> State:
    minimum = min(fitnesses)
    weights = fitnesses if minimum >= 0 else [f - minimum for f
in fitnesses]
    if sum(weights) <= 0:
        return rng.choice(population)
    return rng.choices(population, weights=weights, k=1)[0]
```

2.4.4 Persilangan dan Perbaikan Individu

Persilangan (*crossover*) dilakukan untuk menghasilkan individu baru dengan menggabungkan informasi penempatan paket dari dua induk. Karena hasil persilangan belum tentu memenuhi seluruh *constraints*, diperlukan proses perbaikan (*repair*) sebelum individu digunakan pada generasi berikutnya. Proses ini melibatkan dua metode, yaitu `_crossover()` dan `_repair_placements()`.

- **Metode `_crossover()`**

Metode `_crossover()` digunakan untuk menghasilkan individu baru menggunakan teknik *one-point crossover*. Titik persilangan dipilih secara acak, kemudian informasi penempatan paket sebelum titik tersebut diwarisi dari induk pertama, sedangkan sisanya diwarisi dari induk kedua. Hasil persilangan selanjutnya diproses menggunakan `_repair_placements()` untuk memastikan individu yang dihasilkan valid.

Kode 2.44 Implementasi metode `_crossover()`

```
def _crossover(self, first: State, second: State, rng:
random.Random) -> State:
    package_ids = list(self.problem.package_map)
    if len(package_ids) < 2:
        return first.copy() if rng.random() < 0.5 else
second.copy()
    crossover_point = rng.randint(1, len(package_ids) - 1)
    genes = {
        package_id: deepcopy(first.placements[package_id] if
index < crossover_point else second.placements[package_id])
        for index, package_id in enumerate(package_ids)
    }
    return self._repair_placements(genes, rng)
```

- **Metode `_repair_placements()`**

Metode `_repair_placements()` digunakan untuk memperbaiki konfigurasi penempatan paket hasil persilangan agar memenuhi seluruh *constraints*. Proses dimulai dengan menempatkan seluruh paket di luar truk, kemudian mencoba memasukkan kembali paket yang diwarisi dalam kondisi berada di dalam truk. Urutan penempatan ditentukan berdasarkan ketinggian paket dari yang terendah, kemudian nilai paket dari yang tertinggi. Apabila posisi awal tidak valid, metode mencoba mencari posisi alternatif menggunakan `_find_feasible_placement()`. Paket yang tidak berhasil ditempatkan akan tetap berada di luar truk.

Kode 2.45 Implementasi metode `_repair_placements()`

```
def _repair_placements(self, genes: dict[str, PackagePlacement],
rng: random.Random) -> State:
    state = State({
        package.id: PackagePlacement(None, None,
genes[package.id].orientation)
        for package in self.problem.packages
    })
    inherited_inside = [(package_id, placement) for package_id,
placement in genes.items() if placement.is_inside]
```

```
    inherited_inside.sort(key=lambda item: (item[1].coordinate.z,
-self.problem.package_map[item[0]].value, item[0]))

    for package_id, placement in inherited_inside:
        coordinate = deepcopy(placement.coordinate)
        candidate = self.problem._move_package(state, package_id,
placement.truck_id, coordinate, placement.orientation)
        if candidate is None:
            candidate =
self.problem._find_feasible_placement(state, package_id,
placement.truck_id, [placement.orientation], rng)
        if candidate is not None:
            state = candidate
    if not self.problem.is_state_valid(state):
        raise ValueError("Repair produced an invalid state")
    return state
```

2.4.5 Mutasi Individu

Mutasi (*mutation*) dilakukan untuk meningkatkan keragaman individu dalam populasi dengan menghasilkan perubahan acak pada konfigurasi penempatan paket. Proses ini bertujuan untuk memperluas eksplorasi ruang solusi dan mengurangi kemungkinan algoritma terjebak pada solusi optimum lokal.

Metode `_mutate()` digunakan untuk menghasilkan individu hasil mutasi dengan memanggil fungsi `generate_next_random_state()` pada kelas `Problem`. Fungsi tersebut mencoba menghasilkan *neighbor state* yang valid melalui operasi *swap*, *move*, atau *rotate*. Apabila tidak ditemukan *neighbor state* yang valid, metode mengembalikan salinan individu sebelumnya.

Kode 2. 46 Implementasi metode `_mutate()`

```
def _mutate(self, individual: State, rng: random.Random) ->
State:
    candidate =
self.problem.generate_next_random_state(individual, rng)
    return candidate if candidate is not None else
individual.copy()
```

Probabilitas mutasi ditetapkan sebesar 20% melalui konstanta `MUTATION_RATE = 0.20`. Nilai ini digunakan pada metode `execute()` untuk menentukan apakah setiap individu hasil *crossover* akan mengalami mutasi.

Pemilihan probabilitas 20% merupakan pendekatan heuristik untuk menyeimbangkan eksplorasi solusi baru dengan upaya mempertahankan karakteristik individu hasil *crossover*. Probabilitas yang terlalu rendah dapat membatasi keragaman populasi, sedangkan probabilitas yang terlalu tinggi

berpotensi mengganggu pewarisan karakteristik individu yang baik. Dengan probabilitas 20%, setiap individu hasil *crossover* memiliki peluang sebesar 20% untuk menjalani proses mutasi.

2.4.6 Eksekusi Algoritma *Genetic Algorithm*

Eksekusi *Genetic Algorithm* dilakukan melalui metode `execute()` yang mengintegrasikan seluruh tahapan pencarian solusi. Proses dimulai dengan pembangkitan populasi awal dan perhitungan nilai *fitness* setiap individu. Selanjutnya, pada setiap iterasi dilakukan seleksi induk, persilangan, dan mutasi untuk membentuk populasi baru. Proses ini berlangsung hingga mencapai jumlah iterasi yang ditentukan oleh parameter `max_iterations`.

Metode `execute()` digunakan untuk menjalankan keseluruhan proses *Genetic Algorithm*. Pada setiap iterasi, metode mencatat nilai *fitness* maksimum dan rata-rata populasi untuk memantau perkembangan kualitas solusi. Selain itu, metode menyimpan individu terbaik sepanjang pencarian, meskipun individu tersebut tidak lagi berada dalam populasi generasi terakhir. Setelah seluruh iterasi selesai, hasil pencarian dikembalikan dalam objek `GeneticAlgorithmResult` yang memuat *state* awal dan akhir, riwayat *fitness*, durasi eksekusi, parameter algoritma, serta solusi terbaik yang pernah ditemukan.

Kode 2.47 Implementasi metode `execute()`

```
def execute(self) -> GeneticAlgorithmResult:
    rng = random.Random(self.seed)
    start = time.perf_counter()
    population = self._initialize_population(rng)
    initial_population = [state.copy() for state in population]
    fitnesses = [self._fitness(state) for state in population]

    initial_best_index = max(range(len(population)), key=lambda
i: fitnesses[i])
    initial_state = population[initial_best_index].copy()
    best_state = initial_state.copy()
    best_fitness = fitnesses[initial_best_index]
    max_history = [max(fitnesses)]
    avg_history = [sum(fitnesses) / len(fitnesses)]

    for _ in range(self.max_iterations):
        new_population = []

        while len(new_population) < self.population_size:
            parent_a = self._select_parent(population, fitnesses,
rng)
            parent_b = self._select_parent(population, fitnesses,
```

```
rng)
            offspring = self._crossover(parent_a, parent_b, rng)
            if rng.random() < self.MUTATION_RATE:
                offspring = self._mutate(offspring, rng)
            new_population.append(offspring)

        population = new_population
        fitnesses = [self._fitness(state) for state in
population]
        generation_best_index = max(range(len(population)),
key=lambda i: fitnesses[i])
        if fitnesses[generation_best_index] > best_fitness:
            best_fitness = fitnesses[generation_best_index]
            best_state = population[generation_best_index].copy()
        max_history.append(max(fitnesses))
        avg_history.append(sum(fitnesses) / len(fitnesses))

    final_best_index = max(range(len(population)), key=lambda i:
fitnesses[i])
    final_state = population[final_best_index].copy()
    final_fitness = fitnesses[final_best_index]

    return GeneticAlgorithmResult(
        initial_state=initial_state,
        final_state=final_state,

objective_values_history_per_iteration=max_history.copy(),
        final_objective_value_result=final_fitness,
        duration_seconds=time.perf_counter() - start,
        population_size=self.population_size,
        max_iterations=self.max_iterations,
        max_fitness_history=max_history,
        avg_fitness_history=avg_history,
        initial_population=initial_population,
        final_population=[state.copy() for state in population],
        seed=self.seed,
        best_ever_state=best_state,
        best_ever_fitness=best_fitness,
        problem=self.problem,
    )
```

2.4.7 Hasil dan Analisis Eksperimen

Eksperimen *Genetic Algorithm* dilakukan menggunakan berkas `untuk_laporan.json` yang memuat 3 truk dan 15 paket. Pengujian dibagi menjadi dua skenario, yaitu variasi jumlah iterasi dengan populasi tetap dan variasi jumlah populasi dengan iterasi tetap. Setiap konfigurasi dijalankan sebanyak tiga kali menggunakan *seed* 42, 123, dan 2026. Probabilitas mutasi ditetapkan sebesar 20% pada seluruh percobaan.

2.2.5.1 Pengaruh Jumlah Iterasi

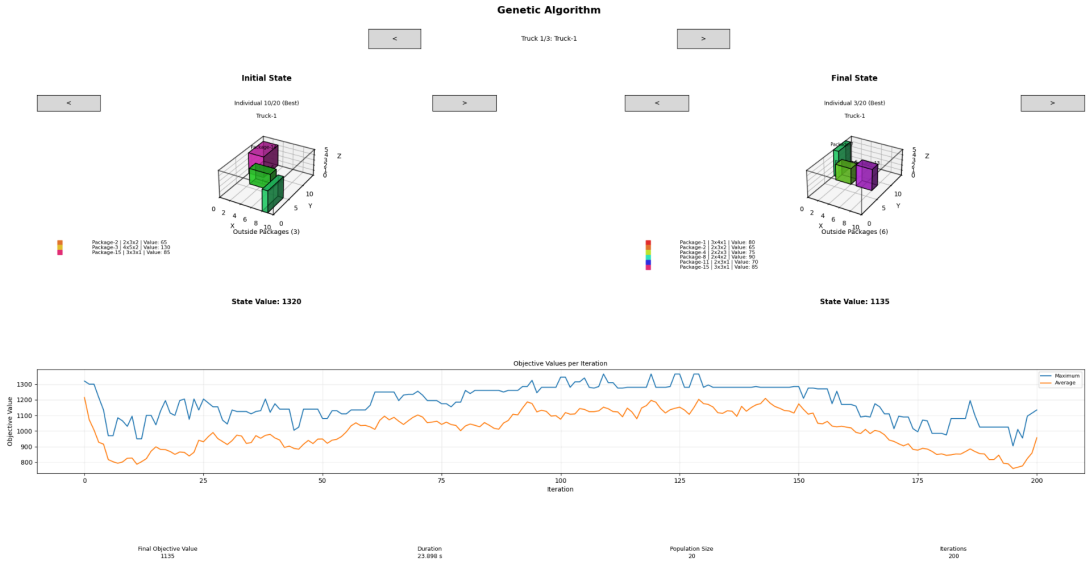
Pengujian pertama dilakukan dengan menetapkan jumlah populasi sebanyak 20 individu dan memvariasikan jumlah iterasi menjadi 50, 100, dan 200.

Tabel 2.4.1 Hasil eksperimen variasi jumlah iterasi (populasi = 20)

Iterasi	Objective Seed 42	Objective Seed 123	Objective Seed 2026	Rata-rata Objective	Durasi Seed 42 (s)	Durasi Seed 123 (s)	Durasi Seed 2026 (s)	Rata-rata Durasi (s)
50	1170	1165	1080	1138,33	8,932	10,347	10,445	9,908
100	1300	1220	1345	1288,33	13,795	14,193	15,814	14,601
200	1170	1130	1135	1145,00	22,947	21,907	23,898	22,917

Berdasarkan Tabel 2.4.1, konfigurasi 100 iterasi menghasilkan rata-rata *objective function* tertinggi, yaitu 1288,33. Sementara itu, konfigurasi 200 iterasi menghasilkan nilai yang lebih rendah, yaitu 1145,00. Hal ini menunjukkan bahwa penambahan iterasi tidak selalu meningkatkan kualitas solusi akhir. Kondisi tersebut dapat terjadi karena individu terbaik tidak dijamin bertahan pada generasi berikutnya.

Dari segi durasi, semakin banyak iterasi yang dijalankan, semakin lama waktu pencarian. Rata-rata durasi meningkat dari 9,908 detik pada 50 iterasi menjadi 22,917 detik pada 200 iterasi.



Gambar 2.4.1 Visualisasi hasil Genetic Algorithm dengan 100 iterasi, populasi 20, dan seed 2026

2.2.5.2 Pengaruh Jumlah Populasi

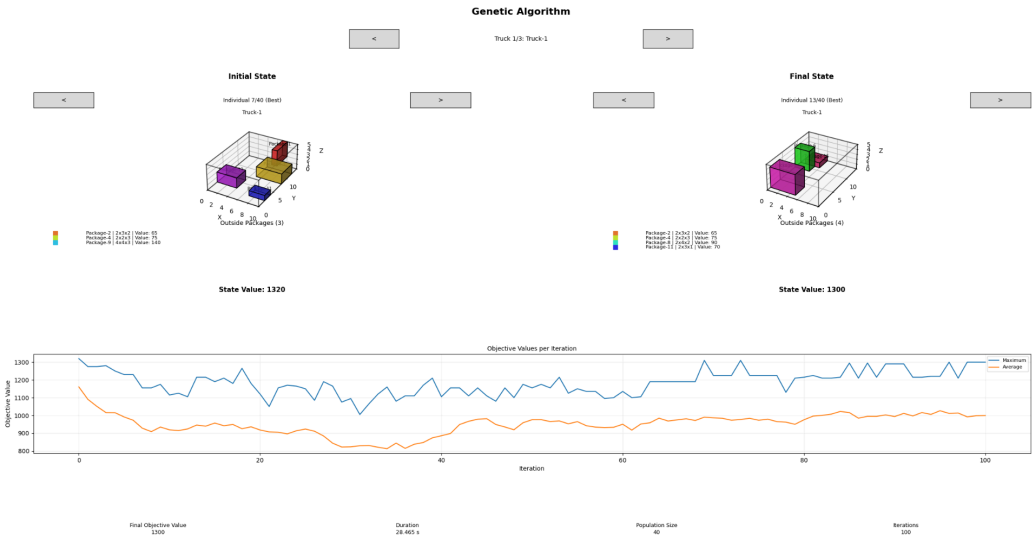
Pengujian kedua dilakukan dengan menetapkan jumlah iterasi sebanyak 100 dan memvariasikan jumlah populasi menjadi 10, 20, dan 40 individu.

Tabel 2.4.2 Hasil eksperimen variasi jumlah populasi (iterasi = 100)

Populasi	Objective Seed 42	Objective Seed 123	Objective Seed 2026	Rata-rata Objective	Durasi Seed 42 (s)	Durasi Seed 123 (s)	Durasi Seed 2026 (s)	Rata-rata Durasi (s)
10	1130	1150	1080	1120,00	6,994	6,984	7,749	7,242
20	1300	1220	1345	1288,33	13,795	14,193	15,814	14,601
40	1240	1300	1290	1276,67	29,034	28,465	29,559	29,019

Berdasarkan Tabel 2.4.2, peningkatan populasi dari 10 menjadi 20 individu meningkatkan rata-rata *objective function* dari 1120,00 menjadi 1288,33. Namun, pada populasi 40 individu, rata-rata nilai sedikit menurun menjadi 1276,67. Dengan demikian, populasi yang lebih besar tidak selalu menghasilkan solusi akhir yang lebih baik.

Peningkatan populasi juga menyebabkan durasi pencarian bertambah. Populasi 40 membutuhkan waktu rata-rata 29,019 detik, hampir dua kali lipat dibandingkan populasi 20. Hal ini disebabkan oleh meningkatnya jumlah individu yang harus diproses pada setiap generasi.



Gambar 2.4.1 Visualisasi hasil Genetic Algorithm dengan 100 iterasi, populasi 40, dan seed 123

2.2.5.3 Analisis Hasil

Berdasarkan seluruh eksperimen, nilai *fitness* maksimum dan rata-rata populasi mengalami fluktuasi sepanjang iterasi, sebagaimana ditunjukkan pada Gambar 2.4.1 dan Gambar 2.4.2. Hal ini berkaitan dengan proses seleksi, *crossover*, dan mutasi yang dapat menghasilkan individu dengan kualitas lebih rendah daripada generasi sebelumnya. Tidak adanya mekanisme *elitism* juga menyebabkan nilai *objective function* akhir dapat lebih rendah daripada nilai awal.

Dari segi konsistensi, konfigurasi 200 iterasi menghasilkan nilai akhir antara 1130–1170, sedangkan konfigurasi 100 iterasi menghasilkan nilai antara 1220–1345. Pada variasi populasi, konfigurasi 40 individu menghasilkan nilai antara 1240–1300, lebih konsisten dibandingkan populasi 20 yang menghasilkan nilai antara 1220–1345. Hal ini menunjukkan bahwa konfigurasi dengan rata-rata nilai tertinggi belum tentu menghasilkan solusi yang paling konsisten.

Secara keseluruhan, konfigurasi 100 iterasi dengan 20 individu menghasilkan rata-rata nilai *objective function* akhir tertinggi pada eksperimen ini. Namun, peningkatan jumlah iterasi maupun populasi tidak selalu meningkatkan kualitas solusi akhir, meskipun keduanya cenderung menambah durasi pencarian.

BAB III
Kesimpulan dan Saran

3.1 Kesimpulan

Dari pengujian yang telah dilakukan, pengujian ini membuktikan bahwa permasalahan ini dapat diselesaikan dengan menggunakan pendekatan *local search*. Hal ini ditunjukkan dengan Simulated Annealing dapat mengeksplorasi state dengan efisien dan sangat mengandalkan kemampuannya untuk dapat menghindari stuck di local optima yang dimampukan dengan menghitung acceptance probabilitynya. Di lain sisi, Hill Climbing memiliki karakteristiknya masing-masing yang menuntunnya ketika melakukan pencarian, misalnya hanya menerima state yang lebih baik, state yang lebih baik atau sama, memilih dari random, ataupun memulai secara random terus menerus. Akan tetapi, Hill Climbing menjadi algoritma yang rawan untuk stuck di local optima, berbeda dengan Simulated Annealing yang memiliki cooling ratenya. Di sisi lain, Genetic Algorithm memiliki karakteristik pencarian yang berdasarkan pada populasi yang mengeksplorasi banyak kandidat solusi melalui seleksi, crossover, dan mutasi. Peningkatan populasi akan meningkatkan konsistensi solusi, tetapi membutuhkan komputasi yang lebih tinggi.

3.2 Saran

Untuk hill climbing, varian random restart akan membantu untuk membuat algoritma ini mengulang ketika ngestuck di local optima. Untuk Simulated Annealing, hasil akan sangat dipengaruhi oleh skema penurunan suhu, suhu awal, dan jumlah iterasi. Dengan demikian perlu dilakukan lebih banyak pengujian menggunakan beragam skema pendinginan agar mendapatkan skema terbaik untuk suatu permasalahan. Untuk *Genetic Algorithm*, parameter input perlu dicoba untuk lebih banyak dituning agar dapat memperoleh kombinasi yang paling efisien. Selain itu juga, secara umum, fungsi-fungsi pembantu yang digunakan haruslah dipastikan melakukan komputasi secara optimal, misalnya di Simulated Annealing ketika akan menghasilkan neighbor baru, dengan demikian meminimalkan komputasi yang diperlukan untuk menghasilkan state baru.

Pembagian Tugas

NIM	Nama	Tugas
13524029	Niko Samuel Simajuntak	Genetic Algorithm, Visualizer, Laporan
13524030	Irvin Tandiarrang Sumual	Simulated Annealing, Visualizer, Laporan
13524033	Ray Owen Martin	Hill Climbing, Visualizer, Laporan

Referensi

[1] https://algorithmafternoon.com/books/simulated_annealing/chapter02/
[2] https://www.w3schools.com/python/matplotlib_intro.asp
[3] Institut Teknologi Bandung, "Beyond Classical Search," materi kuliah IF3170 Inteligensi Artifisial, Semester I Tahun Akademik 2026/2027, 2026.
[4] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Oct. 20, 2026].
[5] https://www.datacamp.com/tutorial/hill-climbing-algorithm-for-ai-in-python
[6] http://terbium.io/2017/12/matplotlib-3d/
[7] Tubes1-naturalstupidity/README.md at main · graceevelyns/Tubes1-naturalstupidity

Lampiran Form Penggunaan Generative AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM	: 1. Niko Samuel Simanjuntak / 13524029 2. Irvin Tandiarrang Sumual / 13524030 3. Ray Owen Martin / 13524033
Fakultas/Sekolah	: STEI - Komputasi
Kode Mata Kuliah	: IF3170
Nama Mata Kuliah	: Inteligensi Artifisial
Judul Tugas/Proposal	: Tugas Besar 1 IF3170 Inteligensi Artifisial 2025/2026

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan (0 - 5)
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	0
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	1
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	1
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	0

Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Niko Samuel Simanjuntak

13524029



Irvin Tandiarrang Sumual

13524030



Ray Owen Martin

13524033

LAMPIRAN LAINNYA

Repository dapat diakses pada https://github.com/NikSamSim/IF3170_G10_BeraniDanTampan